

Edges, Corners, and Blobs

Edge Detection

Slides from Svetlana Lazebnik

Edge Detection



Winter in Kraków photographed by Marcin Ryczek

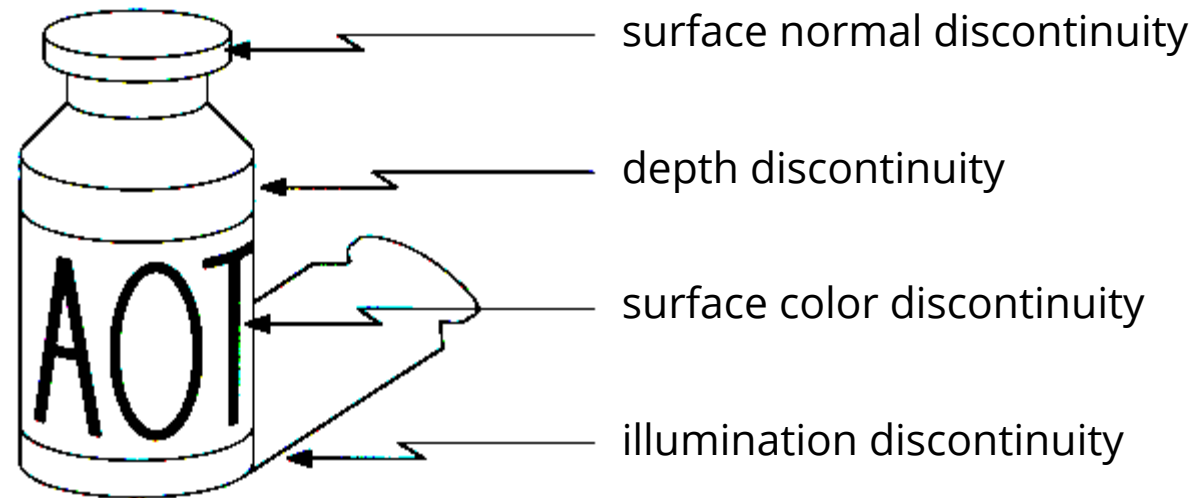
Edge Detection

- **Goal:** Identify sudden changes (discontinuities) in an image
 - Intuitively, most semantic and shape information from the image can be encoded in the edges
 - More compact than pixels
- **Ideal:** artist's line drawing (but artist is also using object-level knowledge)



Origin of Edges

Edges are caused by a variety of factors:

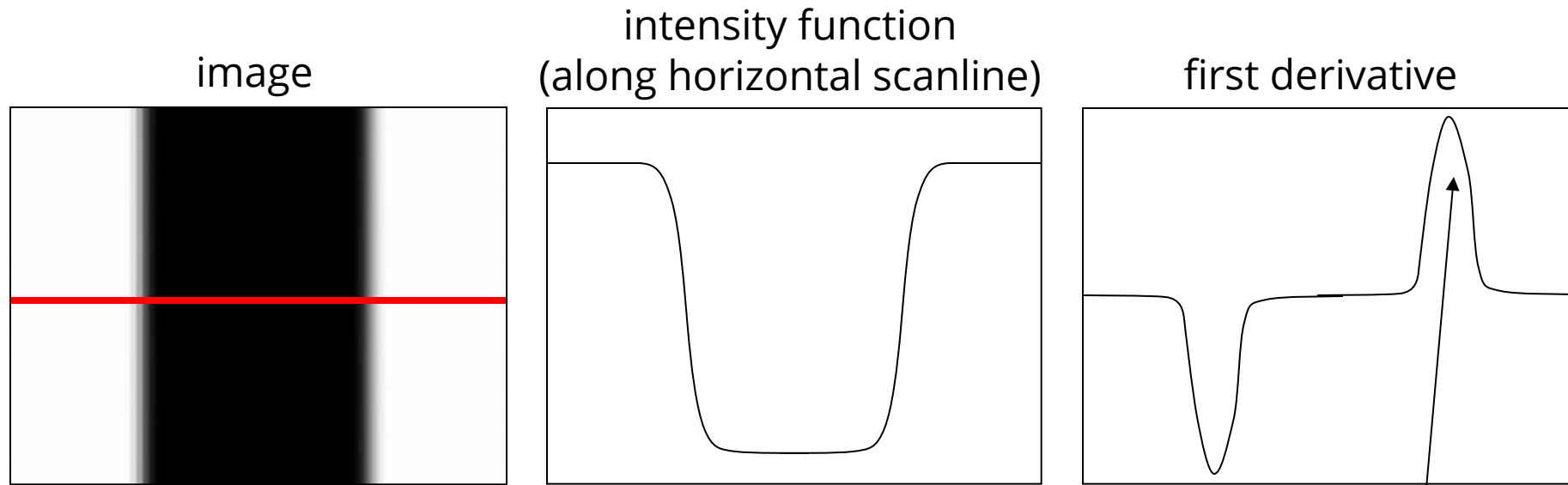


Review: Linear Filtering

- Image blurring — Averaging / Gaussian
- Separability — of Gaussian
- Median filtering — salt pepper noise

Edge Detection

An edge is a place of rapid change in the image intensity function



edge is an intensity discontinuity

edges correspond to
extrema of derivative

Derivatives with Convolution

forward diff $\begin{bmatrix} -1 \\ 1 \end{bmatrix}$
central diff $\begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$

For 2D function $f(x,y)$, the partial derivative is:

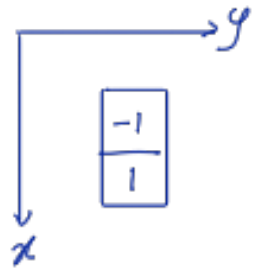
$$\frac{\partial f(x, y)}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}$$

Smallest epsilon is 1 since discrete

For discrete data, we can approximate using finite differences:

$$\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}$$

forward difference formula

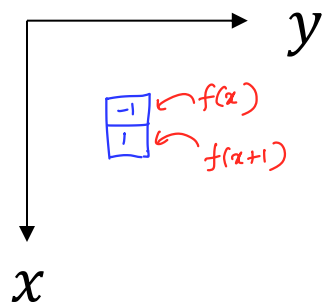


To implement the above as convolution, what would be the associated filter?

But we do not use it as it doesn't have a well defined center.
So we use central difference

Partial Derivatives of an Image

forward
difference
kernel



$$\frac{\partial f(x, y)}{\partial y}$$

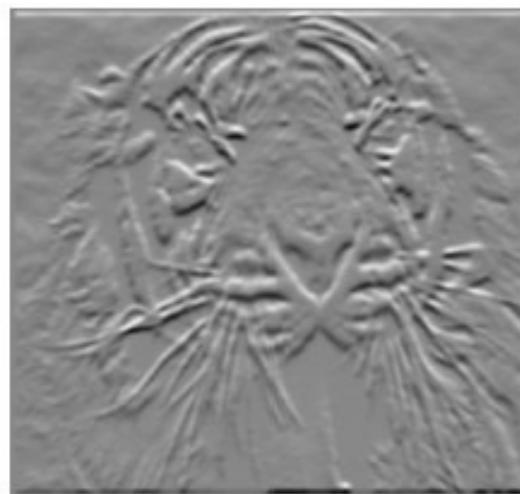
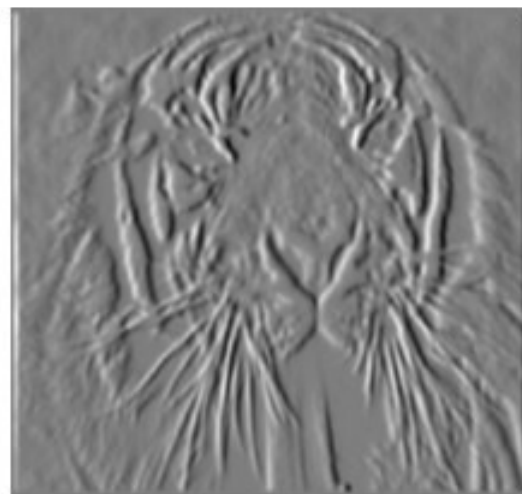
$$\begin{bmatrix} -1 & 1 \end{bmatrix}$$



$$\begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$$

This looks like area like Dark Bright
so it differentiate in the
↓ direction

$$\frac{\partial f(x, y)}{\partial x}$$



$$\begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

or

$$\begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

This is better
* central
difference
formula

another approach

$$\begin{bmatrix} 1 & -2 & 1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Which shows changes with respect to x ?

Finite Difference Filters

Other approximations of derivative filters

Prewitt

$$M_x = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Sobel

$$M_x = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

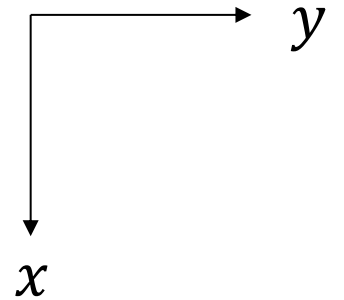
$M_y =$

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$M_y =$

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

improvement since it takes neighbours too.



all some to 0

improve

$$\begin{pmatrix} x \\ 2 \\ x \end{pmatrix}_{3 \times 1} \begin{pmatrix} x & 0 & x \end{pmatrix}_{1 \times 3}$$

give more importance to middle

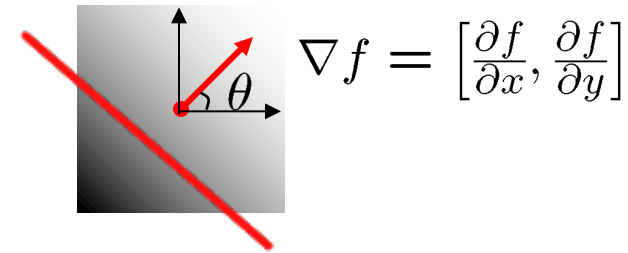
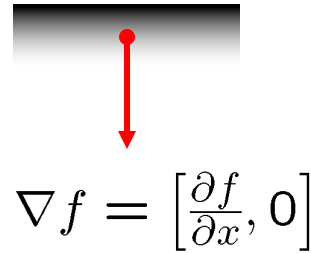
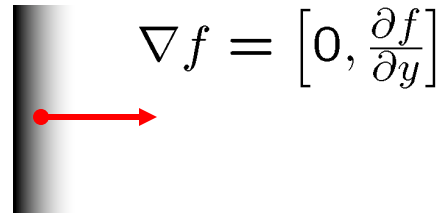
* but averaging filters sum to 1

all values in gradient kernel should sum to zero

Image Gradient

edge direction and gradient direction is $\perp$

The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$



The gradient points in the direction of most rapid increase in intensity

- How does this direction relate to the direction of the edge?

perpendicular

The gradient direction is given by $\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

The edge strength is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}$$

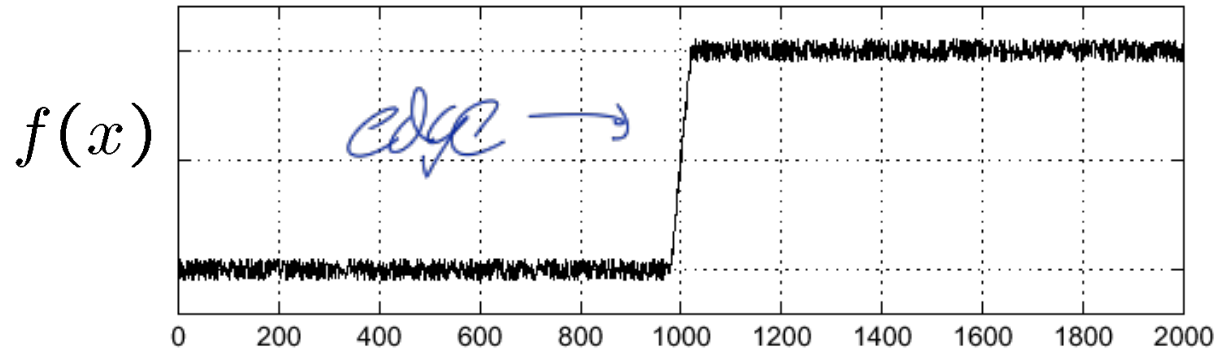
Application: Gradient-Domain Image Editing

Goal: solve for pixel values in the target region to match gradients of the source region while keeping background pixels the same



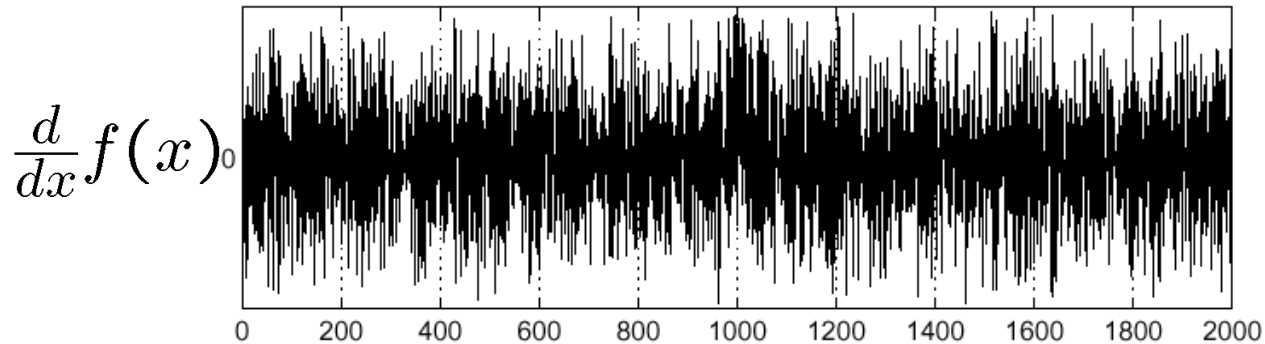
Effects of Noise

Consider a single row or column of the image *(1-D image in the x dir)*



$$f(t) = \sin \omega t$$
$$f'(t) = \omega \cdot \cos \omega t$$

↑
high noise

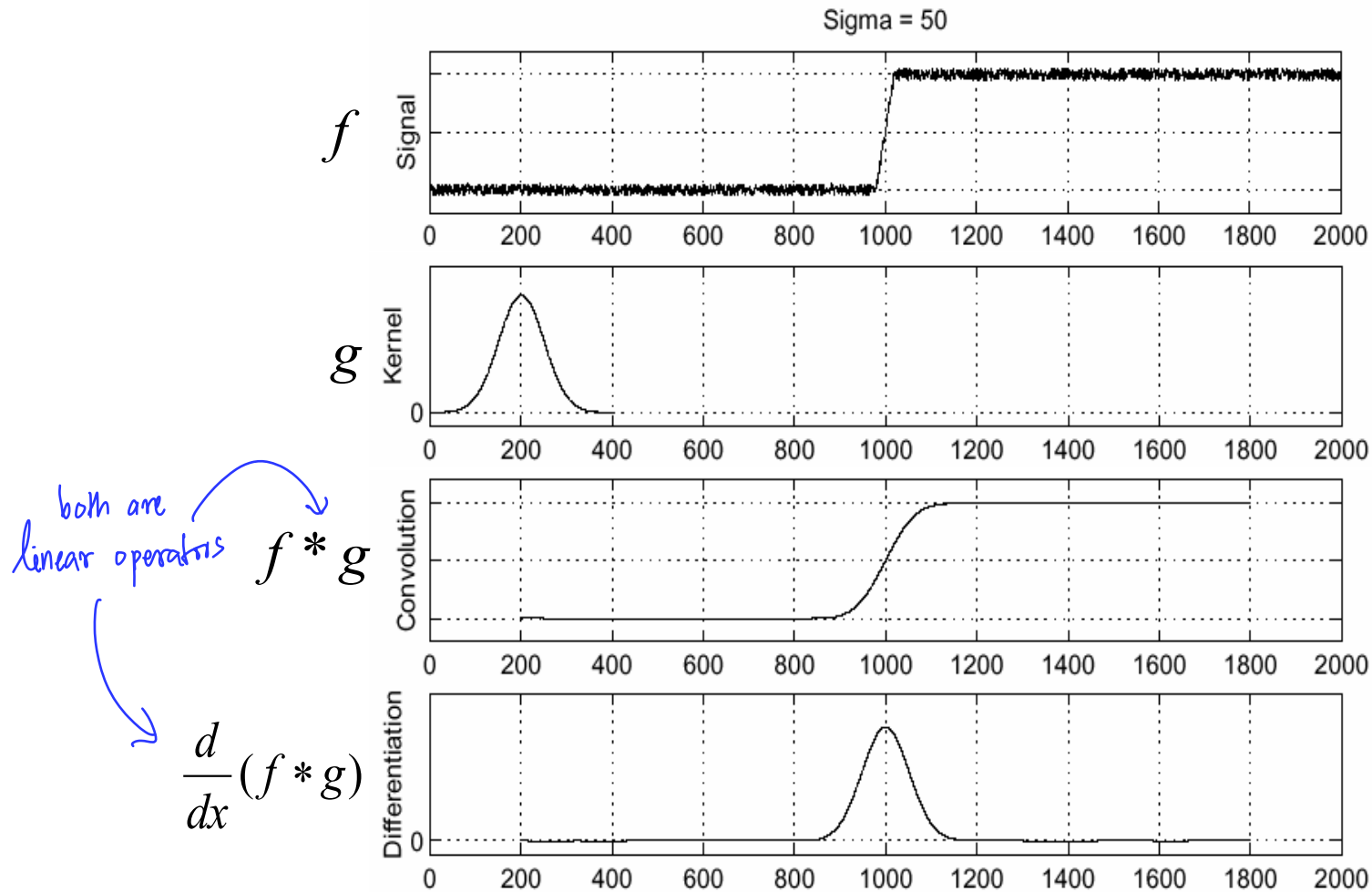


cannot see the edge
with noise.
no

Where is the edge?

Solution: Smooth First

Solution for
detecting edges
in noisy images



Gaussian kernel

smoothed image

now we can see
the image.

To find edges, look for peaks in $\frac{d}{dx}(f * g)$

Derivative Theorem of Convolution

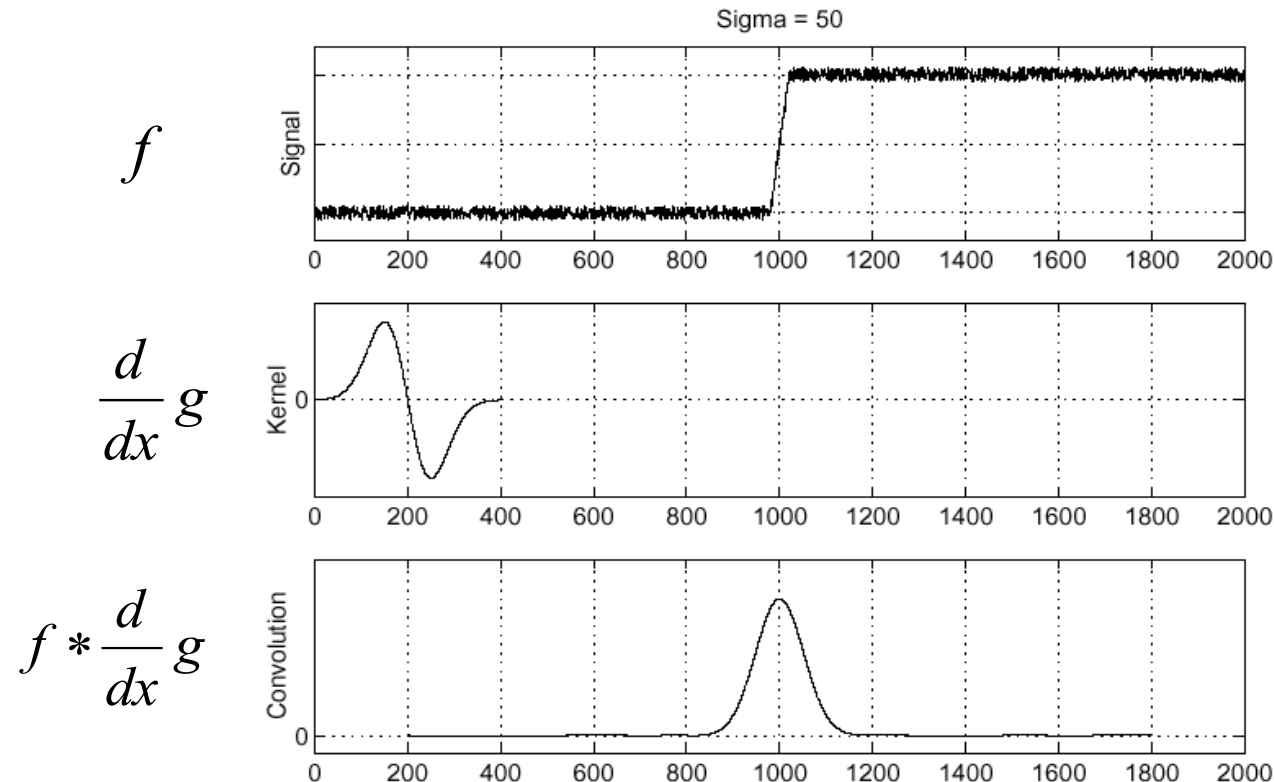
- Differentiation is convolution, and convolution is associative:
- This saves us one operation:

$$\frac{d}{dx}(f * g) = f * \frac{d}{dx}g$$

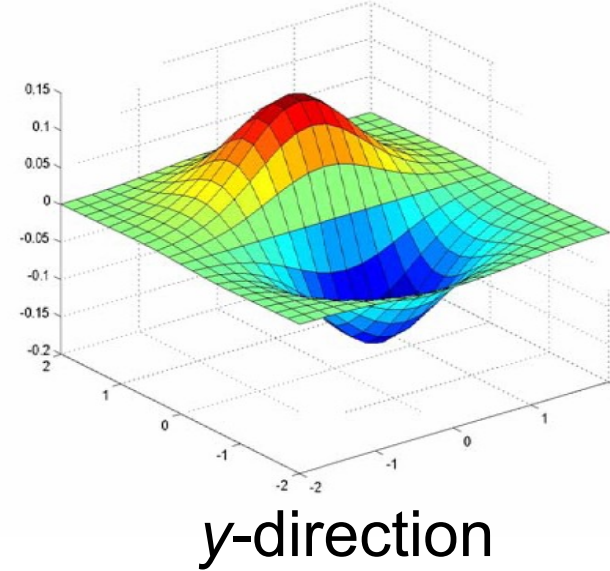
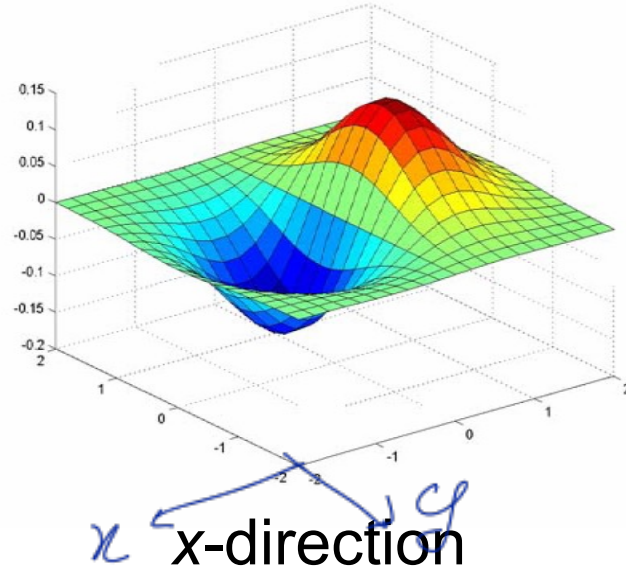
Handwritten red annotations:
 Above the first $\frac{d}{dx}$: $9h^2 + 9h^2$
 Above the second $\frac{d}{dx}$: $9h^2 + 3 \times 3 \times 3 \times 3$

Handwritten blue notes:
 image f : $n \times n$
 g kernel: 3×3
 $\frac{d}{dx}$ kernel: 3×3

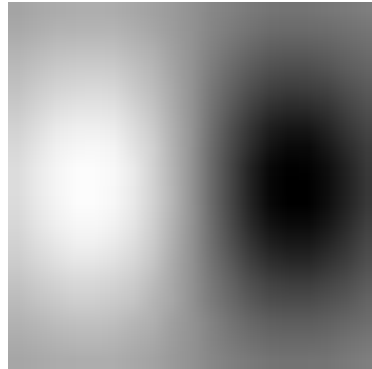
Handwritten blue note:
 due to these are linear operators.



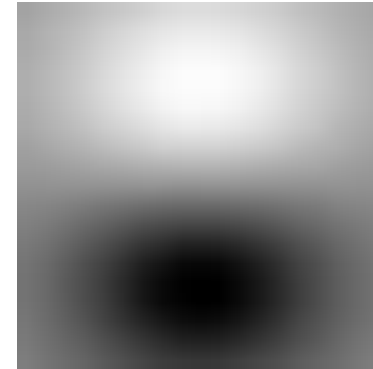
Derivative of Gaussian Filters



vertical
edges

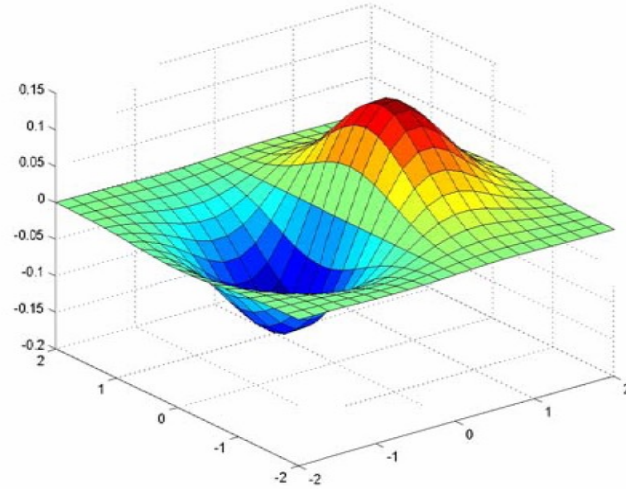


horizontal
edges

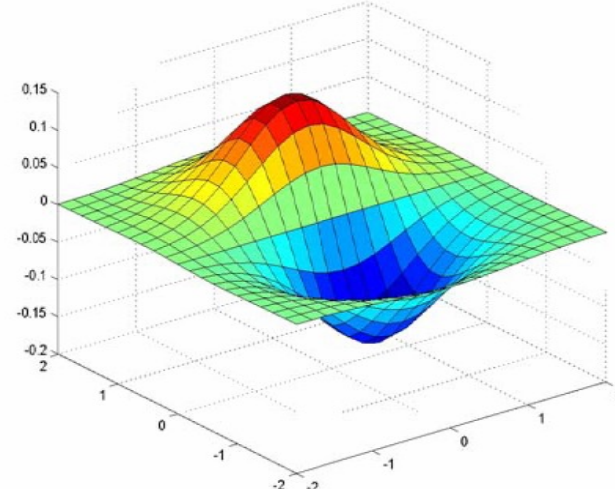
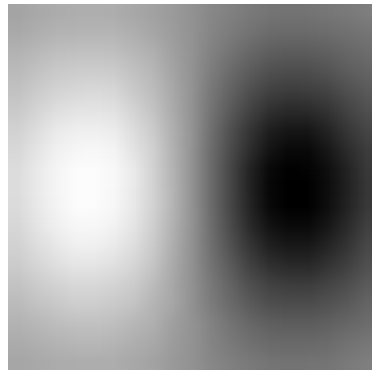


Which one finds horizontal/vertical edges?

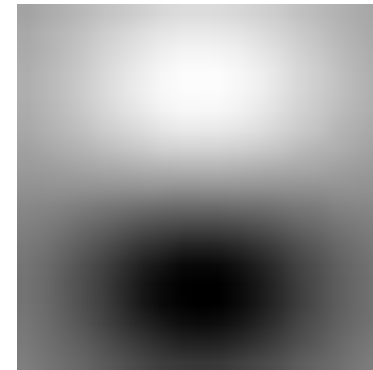
Derivative of Gaussian Filters



x-direction



y-direction



✓ looks much like
Sobel kernel

Are these filters separable?

Recall: Separability of the Gaussian Filter

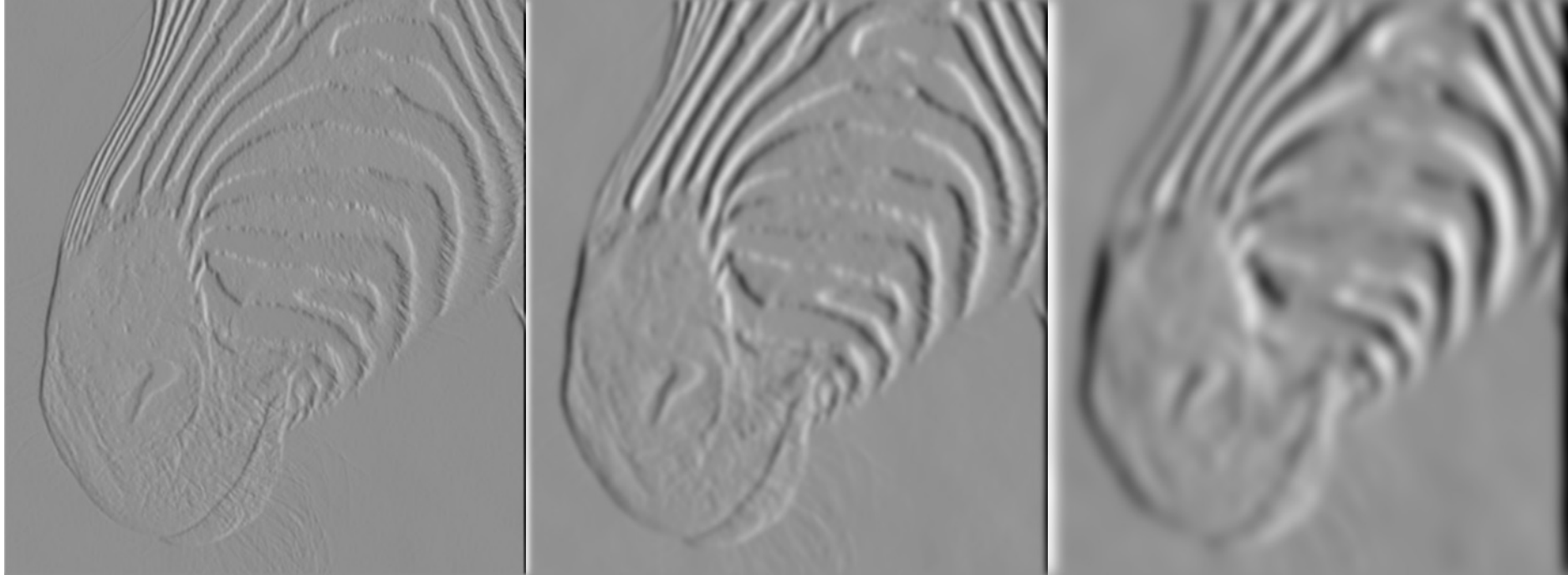
$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp^{-\frac{x^2 + y^2}{2\sigma^2}} \\ &= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{x^2}{2\sigma^2}} \right) \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{y^2}{2\sigma^2}} \right) \end{aligned}$$

Handwritten note: 2-D gaussian to 1-D

The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian

Scale of Gaussian Derivative Filter



1 pixel

3 pixels

7 pixels

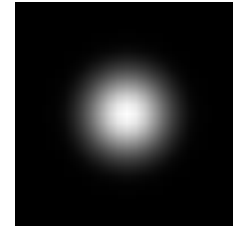
Smoothed derivative removes noise, but blurs edge. Also finds edges at different "scales"

by changing σ we can decide the level of edge

Review: Smoothing vs. Derivative Filters

Smoothing filters

- Gaussian: remove “high-frequency” components; “low-pass” filter
- Can the values of a smoothing filter be negative?
- What should the values sum to?
 - **One:** constant regions are not affected by the filter

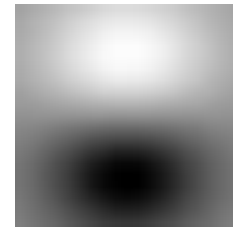
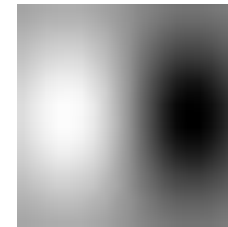


can ?

no

Derivative filters

- Derivatives of Gaussian *or sobel*
- Can the values of a derivative filter be negative?
- What should the values sum to? *(sum of filter)*
 - **Zero:** no response in constant regions
- High absolute value at points of high contrast



can ?

yes

$|x|$ is high

Filtering must be a int operation.
we don't want gaussian kernel to be float
so we first multiply by a large no. to make it
int and do operation
later divide by that number.

The Canny Edge Detector

1. Filter image with derivative of Gaussian
2. Find magnitude and orientation of gradient
3. **Non-maximum suppression:**
 - Thin wide “ridges” down to single pixel width
4. **Linking and thresholding (hysteresis):**
 - Define two thresholds: low and high
 - Use the high threshold to start edge curves and the low threshold to continue them

Thin edges $\Rightarrow$ discard
Thin edges with strong edges $\Rightarrow$ keep

*how we lie
find the peak in the
gradient change*

J. Canny, [**A Computational Approach To Edge Detection**](#), IEEE Trans. Pattern Analysis and Machine Intelligence, 8:679-714, 1986.

The Canny Edge Detector



original image

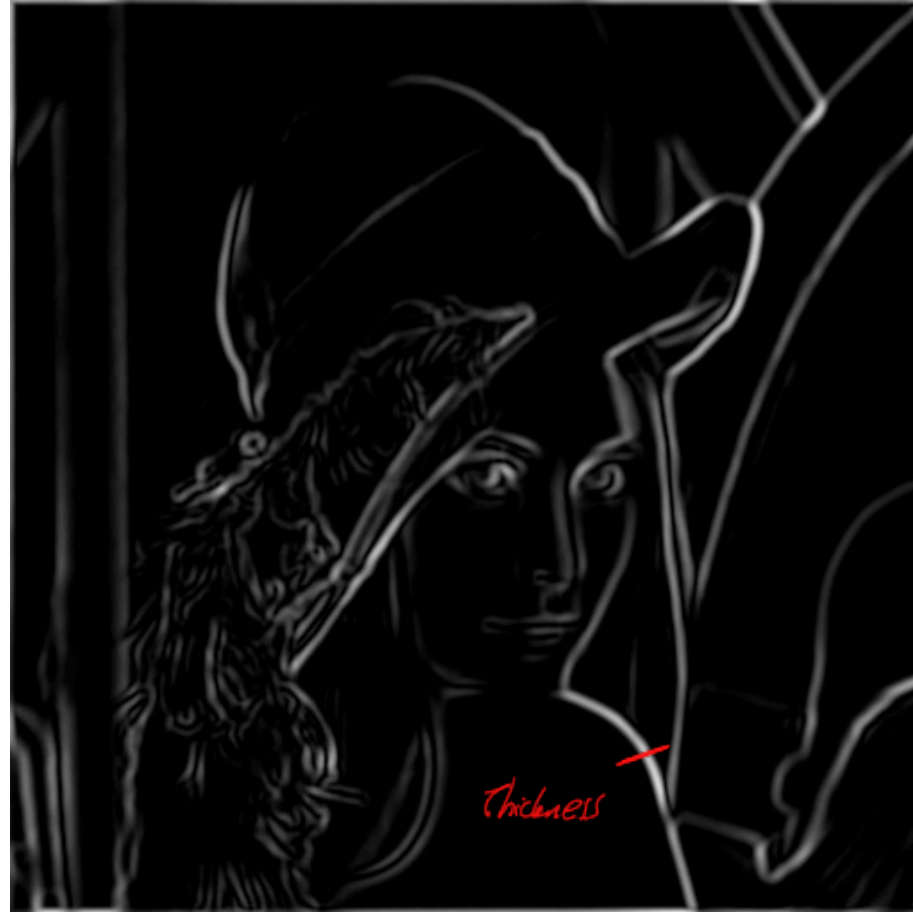
The Canny Edge Detector



$$\sqrt{\frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}}$$

norm of the gradient

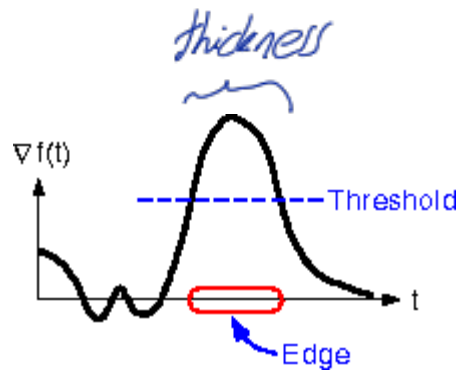
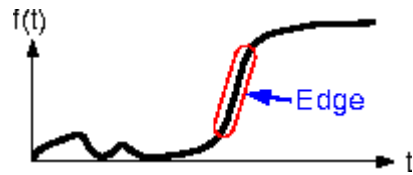
The Canny Edge Detector



thresholding

The Canny Edge Detector

*Dark to bright happens
over a range*

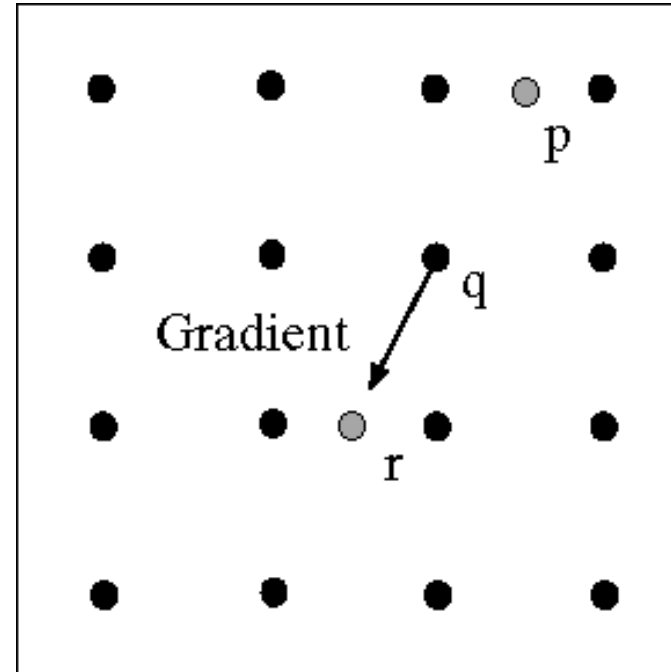
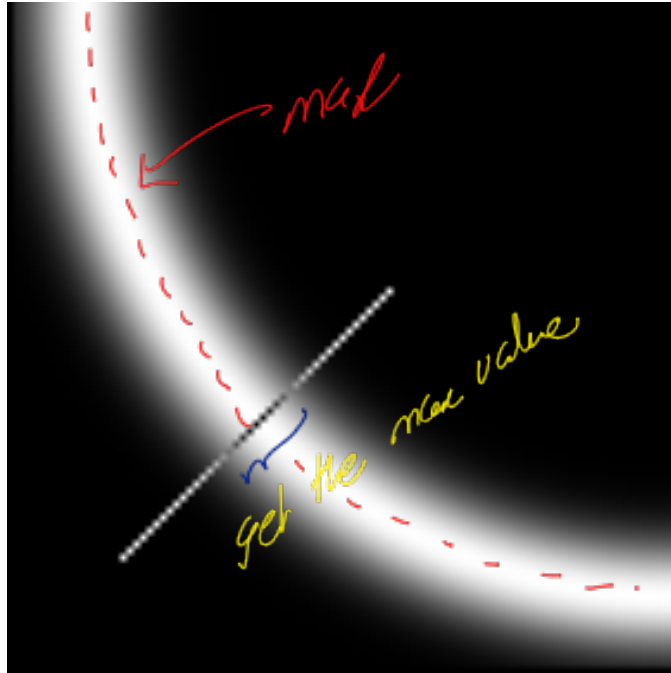


How to turn
these thick
regions of
the gradient
into
curves?

*How to remove
thick line and
get a single line*

thresholding

Non-Maximum Suppression



Check if pixel is local maximum along
gradient direction, select single max across
width of the edge

- requires checking interpolated pixels p and r

The Canny Edge Detector



Problem:
pixels along
this edge
didn't
survive the
thresholding

thinning
(non-maximum suppression)

Hysteresis Thresholding

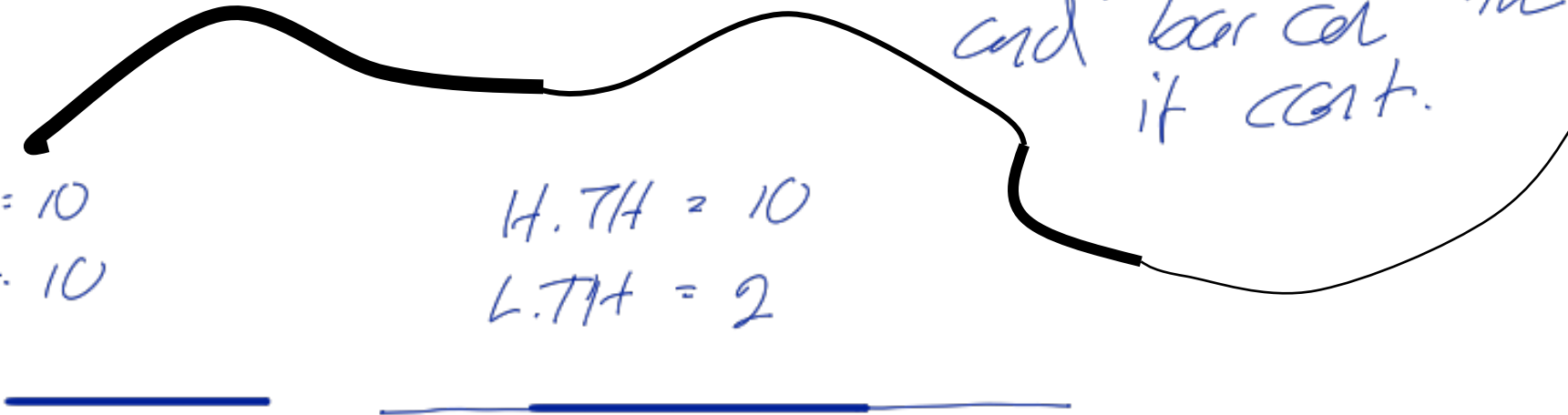
Use a high threshold to start edge curves, and a low threshold to continue them.

$H_TH = \text{strong edges}$
 $L_TH = \text{weak edges}$

Always start at high TH
and lower to the L_TH
if cont.

$H_TH = 10$
 $L_TH = 10$

$H_TH = 10$
 $L_TH = 2$



Hysteresis Thresholding

*Only keep low T_h
edges that are
connected to high T_h
edges*



original image



**high threshold
(strong edges)**



**low threshold
(weak edges)**



hysteresis threshold

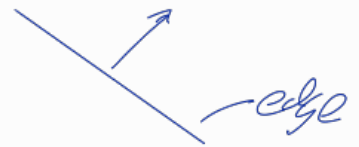
Curve of edge $\rightarrow$ depth
 surface tex
 colour
 illumination

} discontinuity.

Normally to find edge, first derivative of image (sobel) is enough.

Gradient $\rightarrow \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$

Gradient direction $\perp$ to edge



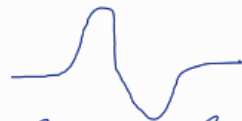
strength of edge is $\sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$

when there is noise we cannot use derivative
 we use gaussian kernel g

$f * g \rightarrow \frac{d(f * g)}{dx}$



Alternatively we can do $f * \frac{d}{dx}(g)$ efficient



can use σ to describe the edge level

• smoothing filters

- always $(+)$

- $\Sigma \Rightarrow 1$

derivative filters

- can be $(-)$

- $\Sigma \Rightarrow 0$

• Canny edge detector steps

1) do $\frac{f}{\partial x}$ & $\frac{f}{\partial y}$ to find edges

2) find gradient and direction

3) Do nms with a TH



4) Do hysteresis TH with a L-TH and H-TH

Only keep L-TH edges if they are a continuation of H-TH edges.

Recap: Canny Edge Detector

1. Compute x and y gradient images
2. Find magnitude and orientation of gradient
- 3. Non-maximum suppression:**
 - Thin wide “ridges” down to single pixel width
- 4. Linking and thresholding (hysteresis):**
 - Define two thresholds: low and high
 - Use the high threshold to start edge curves and the low threshold to continue them



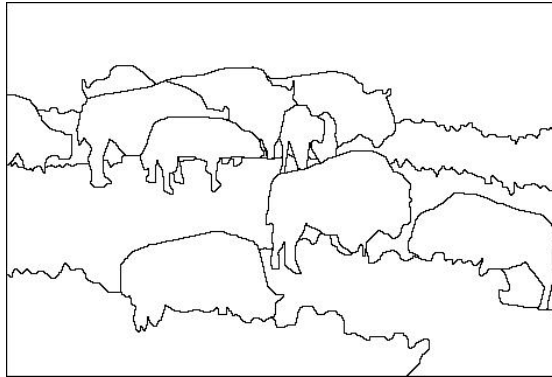
J. Canny, [***A Computational Approach To Edge Detection***](#), IEEE Trans. Pattern Analysis and Machine Intelligence, 8:679-714, 1986.

Image Gradients vs. Meaningful Contours

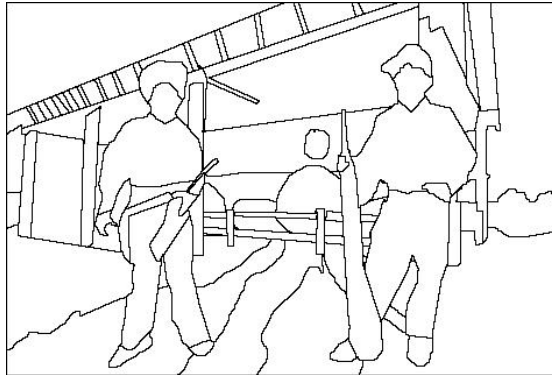
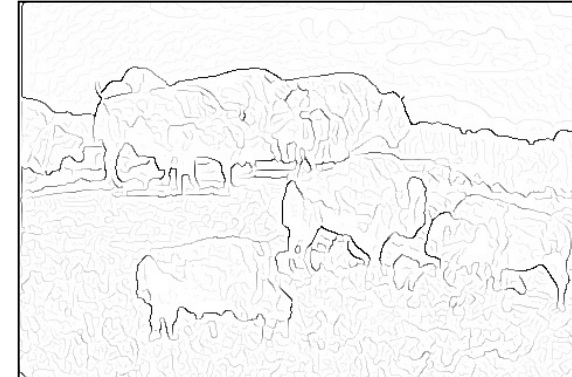
image



human segmentation



gradient magnitude



3

Berkeley segmentation database:

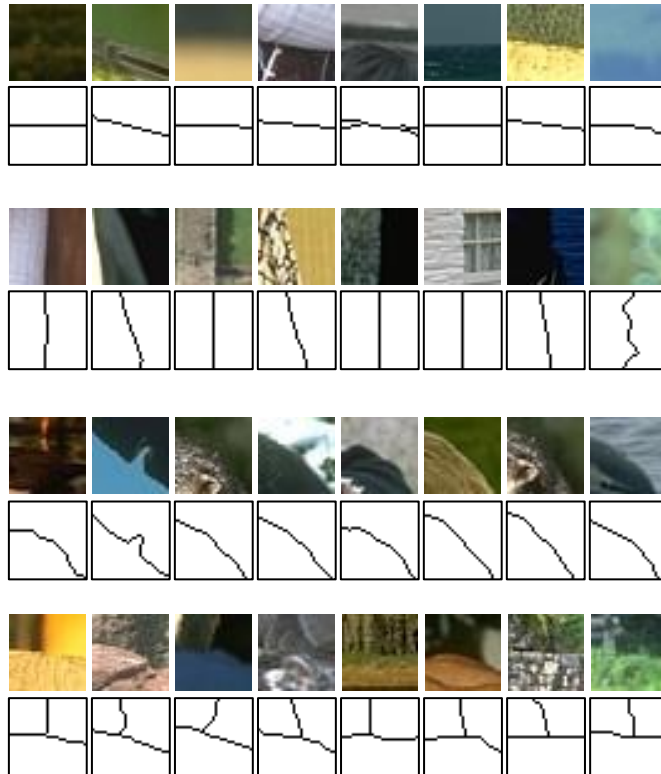
<http://www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/segbench/>

Data-Driven Edge Detection

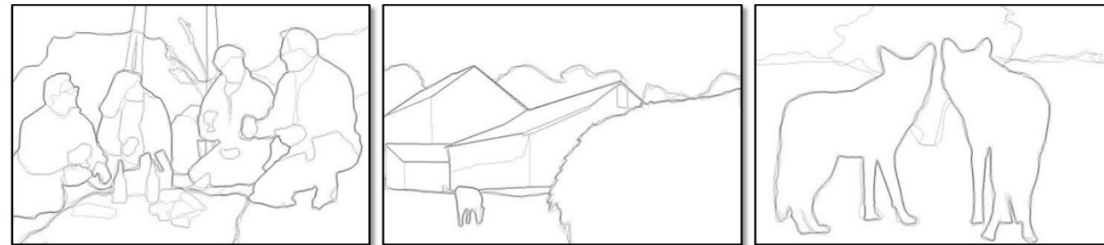
*much better when
using ML model.*

Input images

Training data



Ground truth



Output

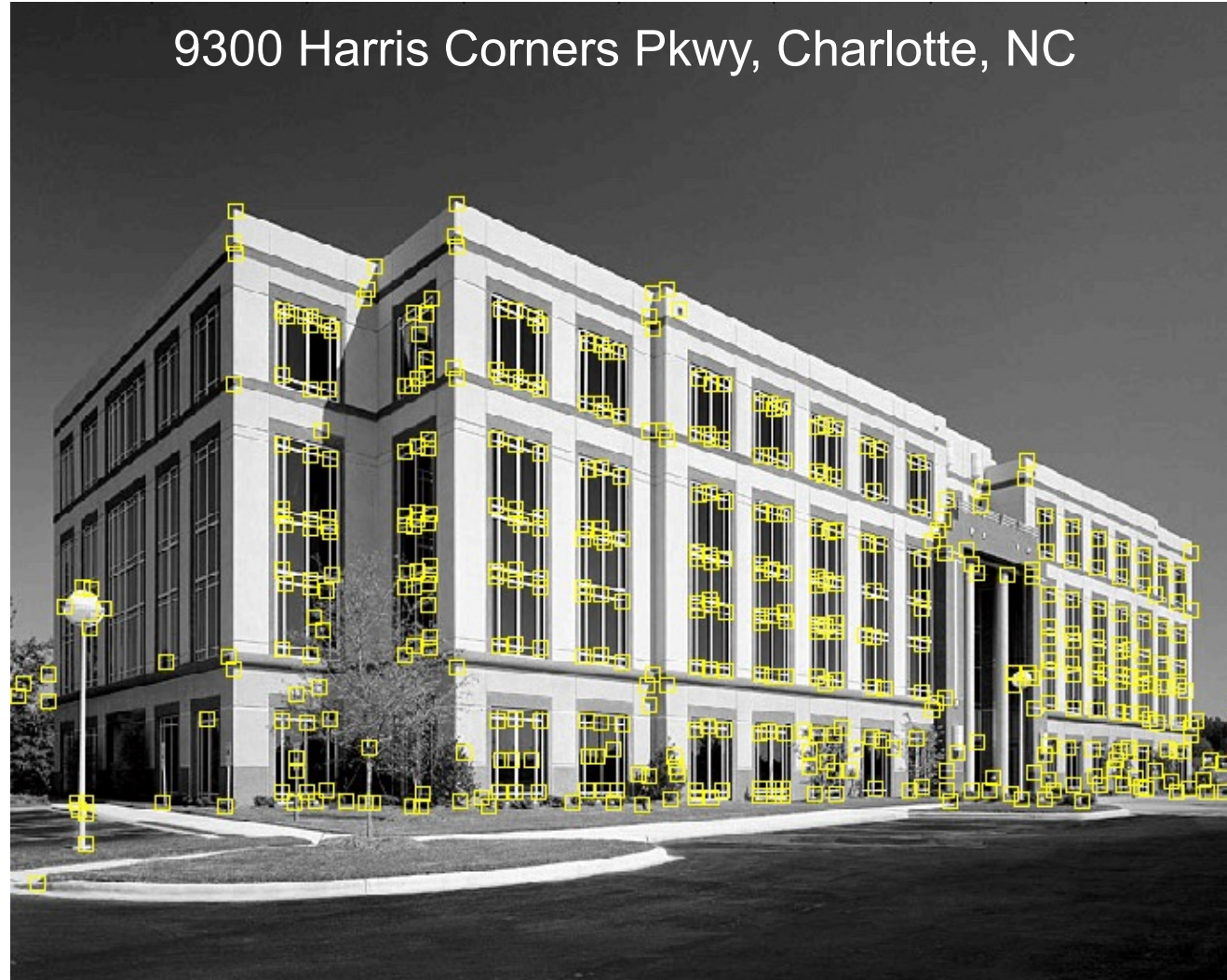


2

Corners

Slides are from Svetlana Lzebnik

Feature Extraction: Corners



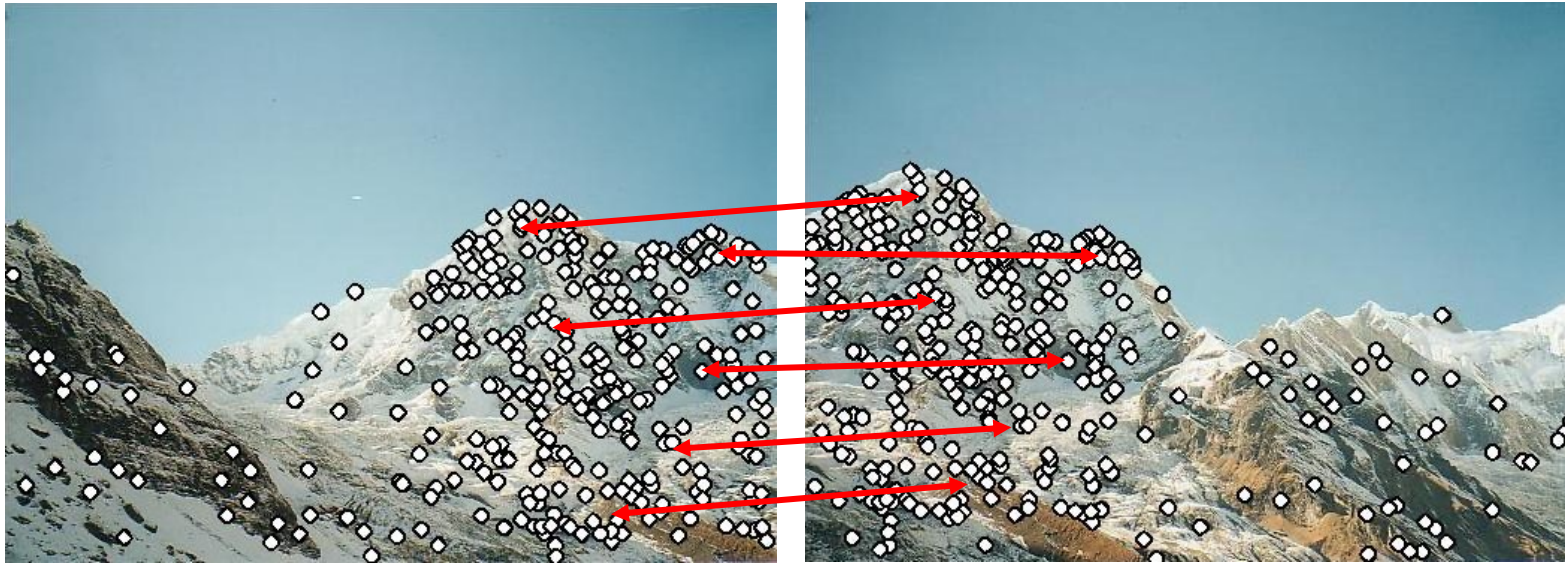
Why Extract Features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Why Extract Features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Step 1: extract features

Step 2: match features

Why Extract Features?

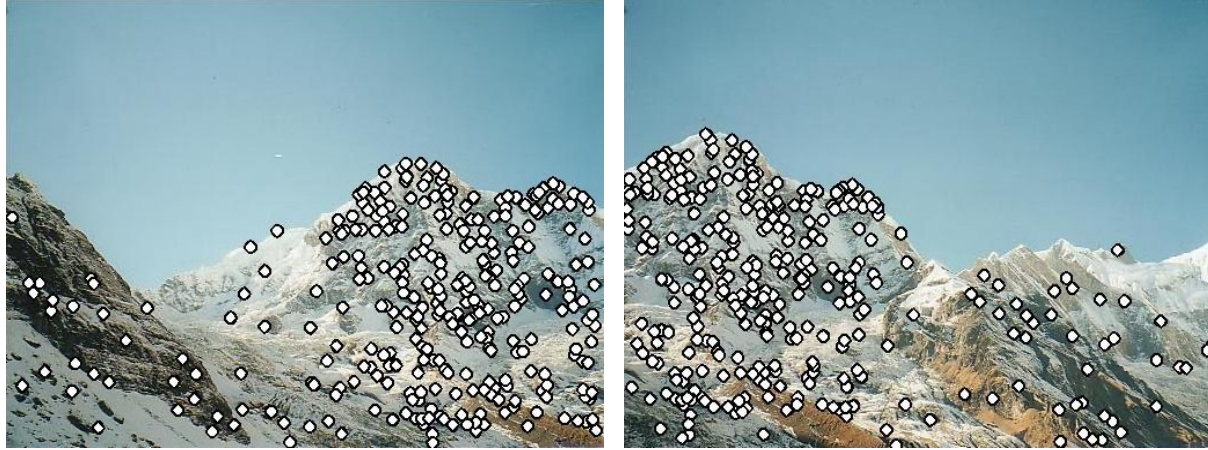
- Motivation: panorama stitching
 - We have two images – how do we combine them?



homography
transformation

- Step 1: extract features
- Step 2: match features
- Step 3: align images

Characteristics of Good Features



- Repeatability
 - The same feature can be found in several images despite geometric and photometric transformations
- Saliency
 - Each feature is distinctive
- Compactness and efficiency
 - Many fewer features than image pixels
- Locality
 - A feature occupies a relatively small area of the image; robust to clutter and occlusion

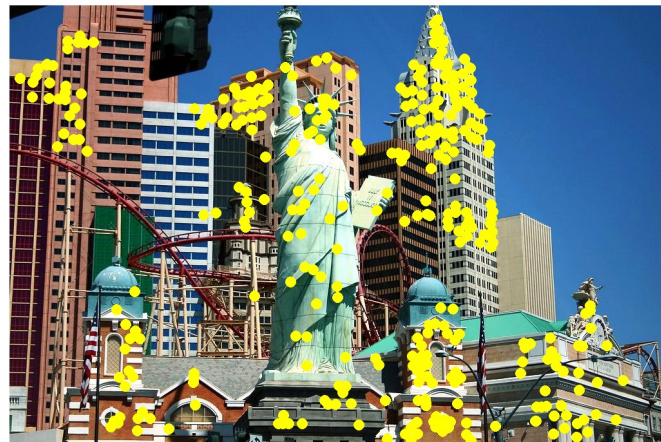
*independent of where
the photo is taken.*

*} Look into these
more*

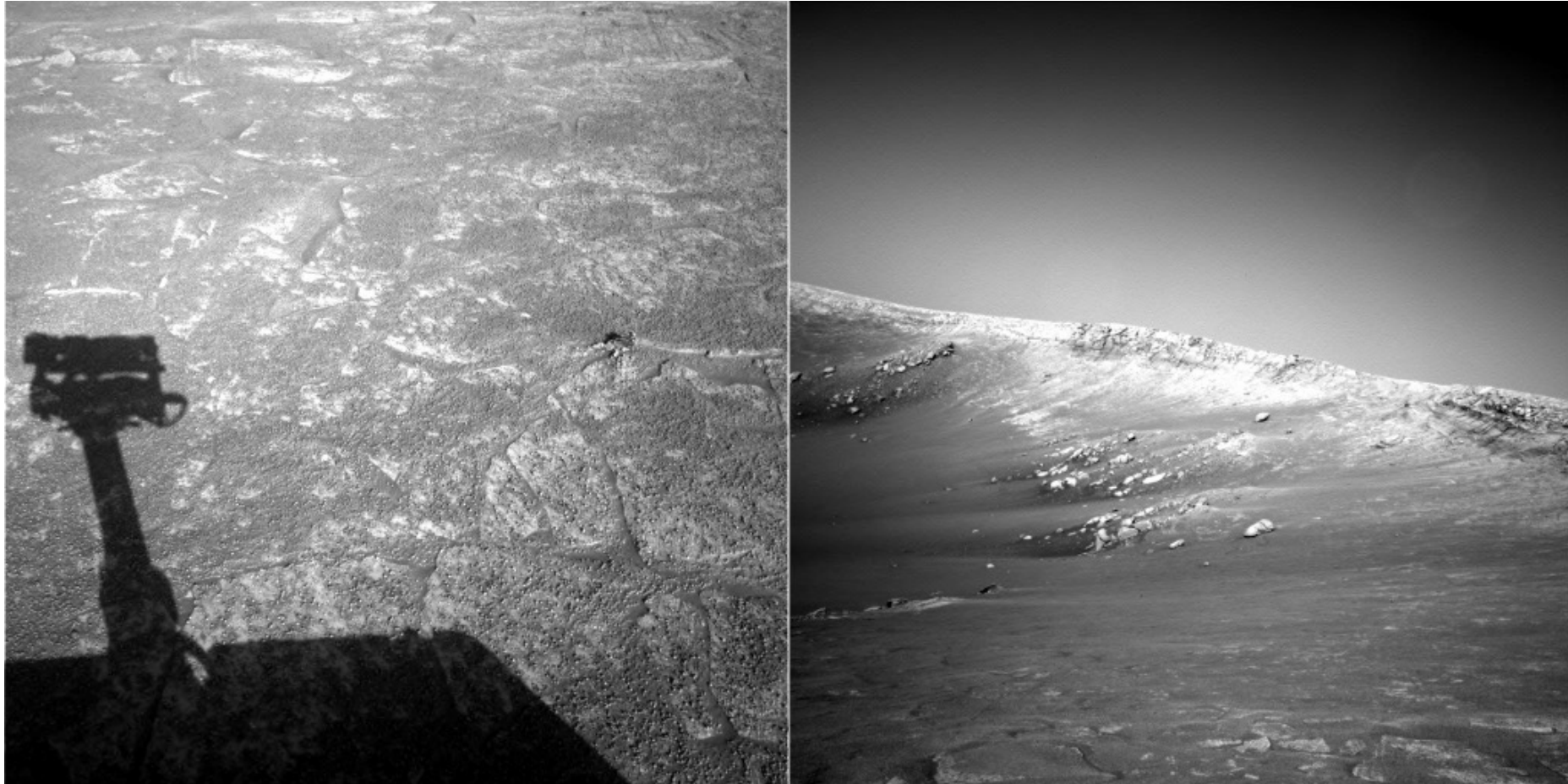
Applications

Feature points are used for:

- Image alignment
- 3D reconstruction
- Motion tracking
- Robot navigation
- Indexing and database retrieval
- Object recognition



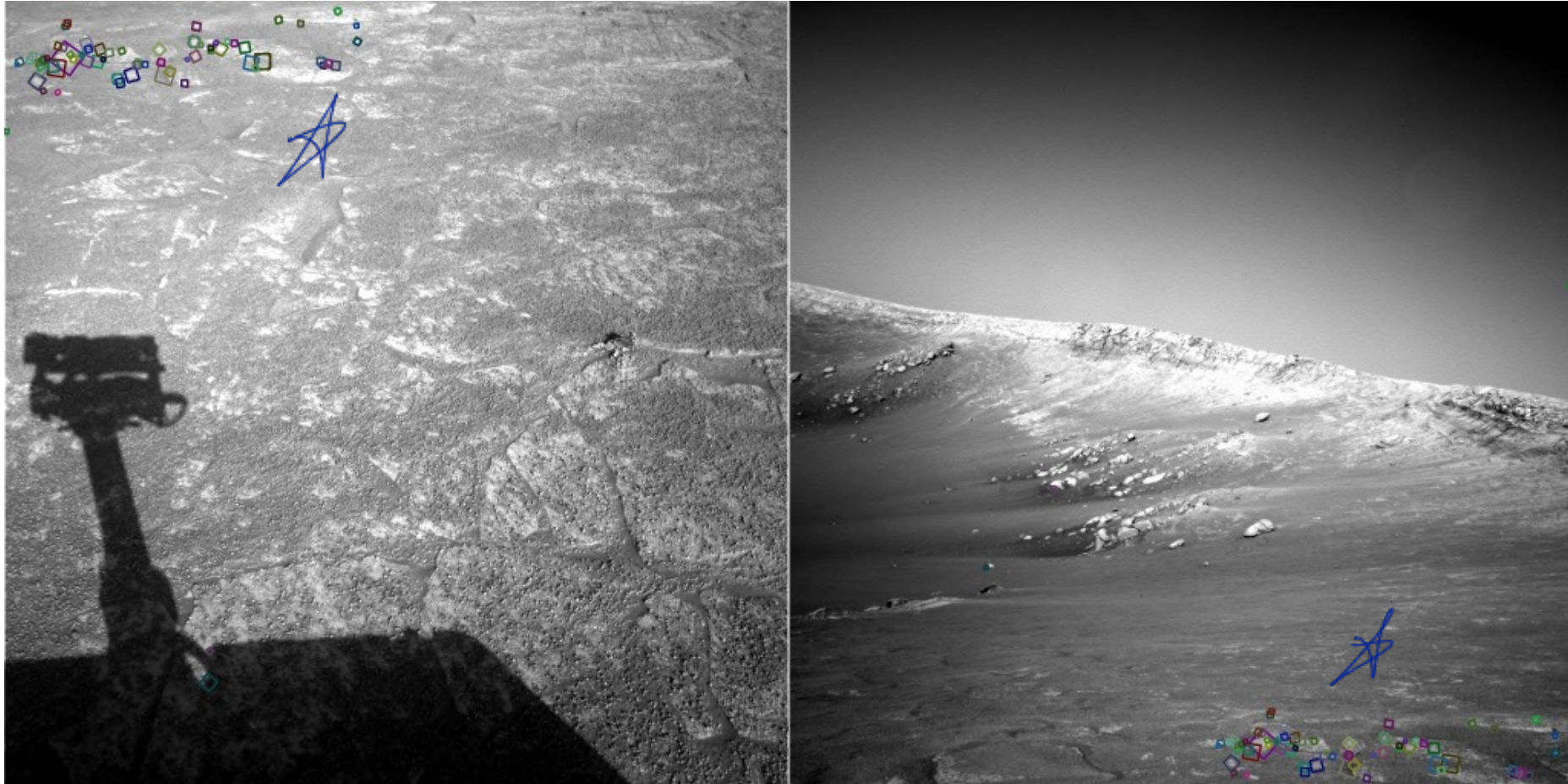
A Hard Feature Matching Problem



NASA Mars Rover images

Answer Below (look for tiny colored squares...)

features can be matched

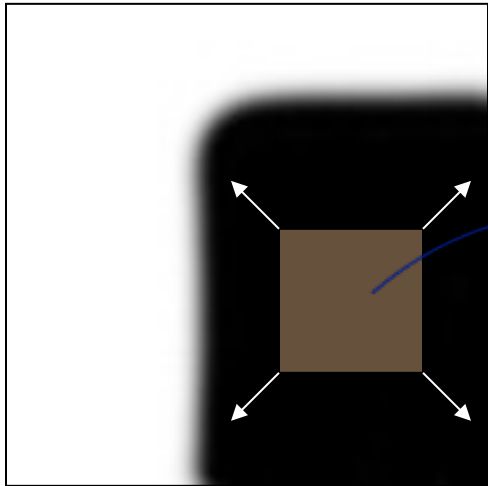


NASA Mars Rover images
with SIFT feature matches
Figure by Noah Snavely

Corner Detection: Basic Idea

We should easily recognize the point by looking through a small window

Shifting a window in *any direction* should give a large change in intensity

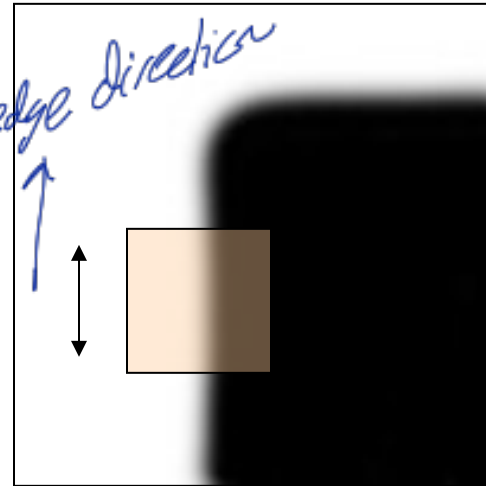


“flat” region:

no change in all
directions

(small change)

0 DoF

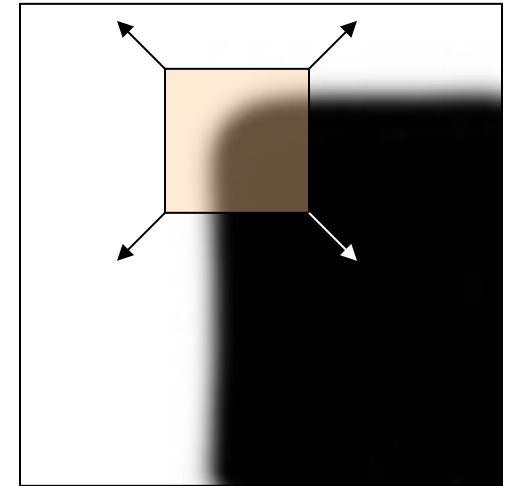


“edge”:

no change
along the edge (vertical)
direction

1 DoF

intensity change only in
one direction (horizontal)



“corner”:

significant
change in all
directions

2 DoF

why do we need corners?

1) Repeatable

2) Salient

3) They are useful into crowd
corners

Corner Detection: Mathematics

take a window image
move a bit and take the window image
subtract
differentiate that.

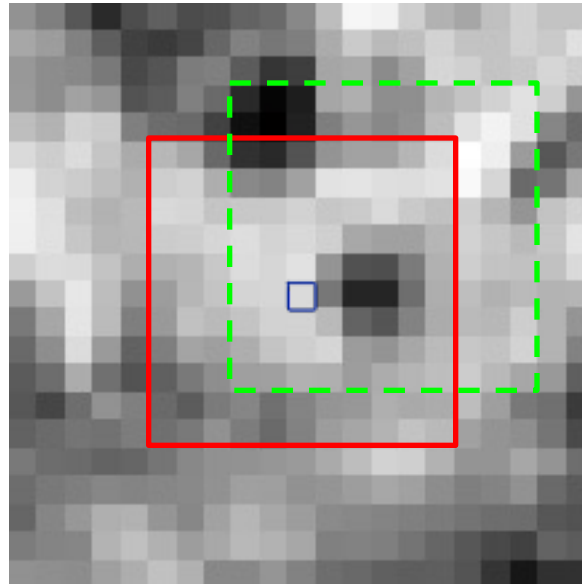
Change in appearance of window W for the shift $[u,v]$:

$$E(u, v) = \sum_{(x,y) \in W} [I(x+u, y+v) - I(x, y)]^2$$

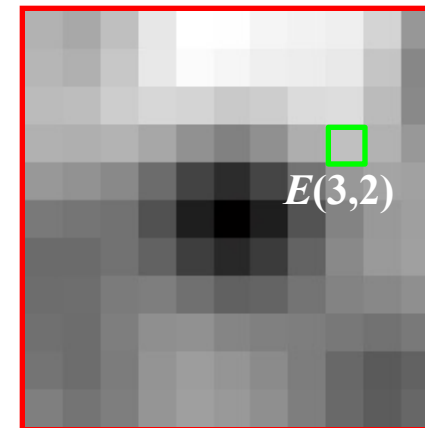
so that
+ and -
will cancel
off

A 3D
plot
for each
point

$I(x, y)$



$E(u, v)$



$E(0,0) = 0$
others have a value

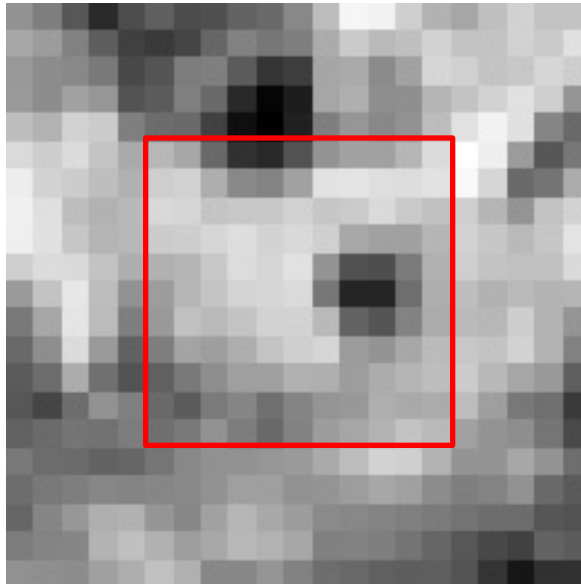
Corner Detection: Mathematics

Change in appearance of window W for the shift $[u,v]$:

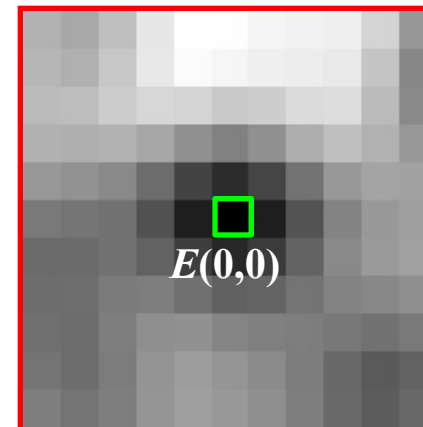
$$E(u, v) = \sum_{(x,y) \in W} [I(x+u, y+v) - I(x, y)]^2$$

But had to do

$I(x, y)$



$E(u, v)$



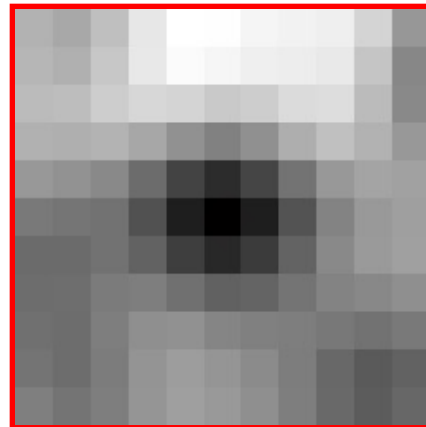
Corner Detection: Mathematics

Change in appearance of window W for the shift $[u,v]$:

$$E(u, v) = \sum_{(x,y) \in W} [I(x+u, y+v) - I(x, y)]^2$$

We want to find out how this function behaves for small shifts

$E(u, v)$



Taylor in 1-D

$$f(x+h) = f(x) + h f'(x) + \frac{h^2}{2} f''(x) + \dots$$

in 2-D since u, v is small.

$$f(x+u, y+v) = f(x, y) + \int f_x(x, y) + \int f_y(x, y)$$

Corner Detection: Mathematics

- First-order Taylor approximation for small motions $[u, v]$:

$$I(x+u, y+v) \approx I(x, y) + I_x u + I_y v$$

- Let's plug this into $E(u, v)$:

from Taylor.


$$\begin{aligned} E(u, v) &= \sum_{(x, y) \in W} [I(x+u, y+v) - I(x, y)]^2 \\ &\approx \sum_{(x, y) \in W} [I(x, y) + I_x u + I_y v - I(x, y)]^2 \\ &= \sum_{(x, y) \in W} [I_x u + I_y v]^2 = \sum_{(x, y) \in W} I_x^2 u^2 + 2I_x I_y uv + I_y^2 v^2 \end{aligned}$$

Corner Detection: Mathematics

The quadratic approximation can be written as

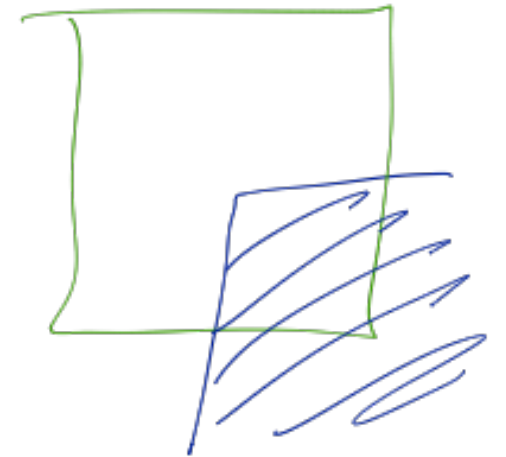
$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

where M is a **second moment matrix** computed from image derivatives:

$$M = \begin{bmatrix} \sum_{x,y} I_x^2 & \sum_{x,y} I_x I_y \\ \sum_{x,y} I_x I_y & \sum_{x,y} I_y^2 \end{bmatrix}$$

(the sums are over all the pixels in the window W)

$$\begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

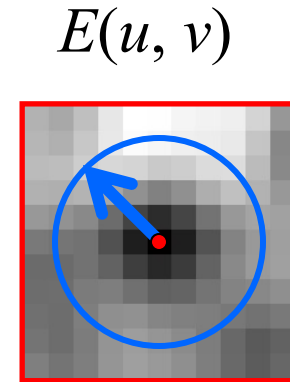


Interpreting the Second Moment Matrix

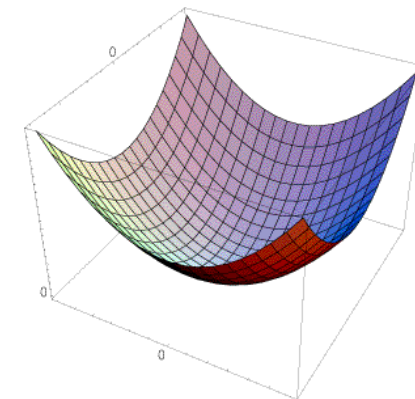
- The surface $E(u,v)$ is locally approximated by a quadratic form. Let's try to understand its shape.
- Specifically, in which directions does it have the smallest/greatest change?

$$E(u, v) \approx [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = \begin{bmatrix} \sum_{x,y} I_x^2 & \sum_{x,y} I_x I_y \\ \sum_{x,y} I_x I_y & \sum_{x,y} I_y^2 \end{bmatrix}$$



*usually
E looks
like this*



Interpreting the Second Moment Matrix

Case 1:

First, consider the axis-aligned case (gradients are either horizontal or vertical)

$$M = \begin{bmatrix} \sum_{x,y} I_x^2 & \sum_{x,y} I_x I_y \\ \sum_{x,y} I_x I_y & \sum_{x,y} I_y^2 \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}$$

→ it is an edge

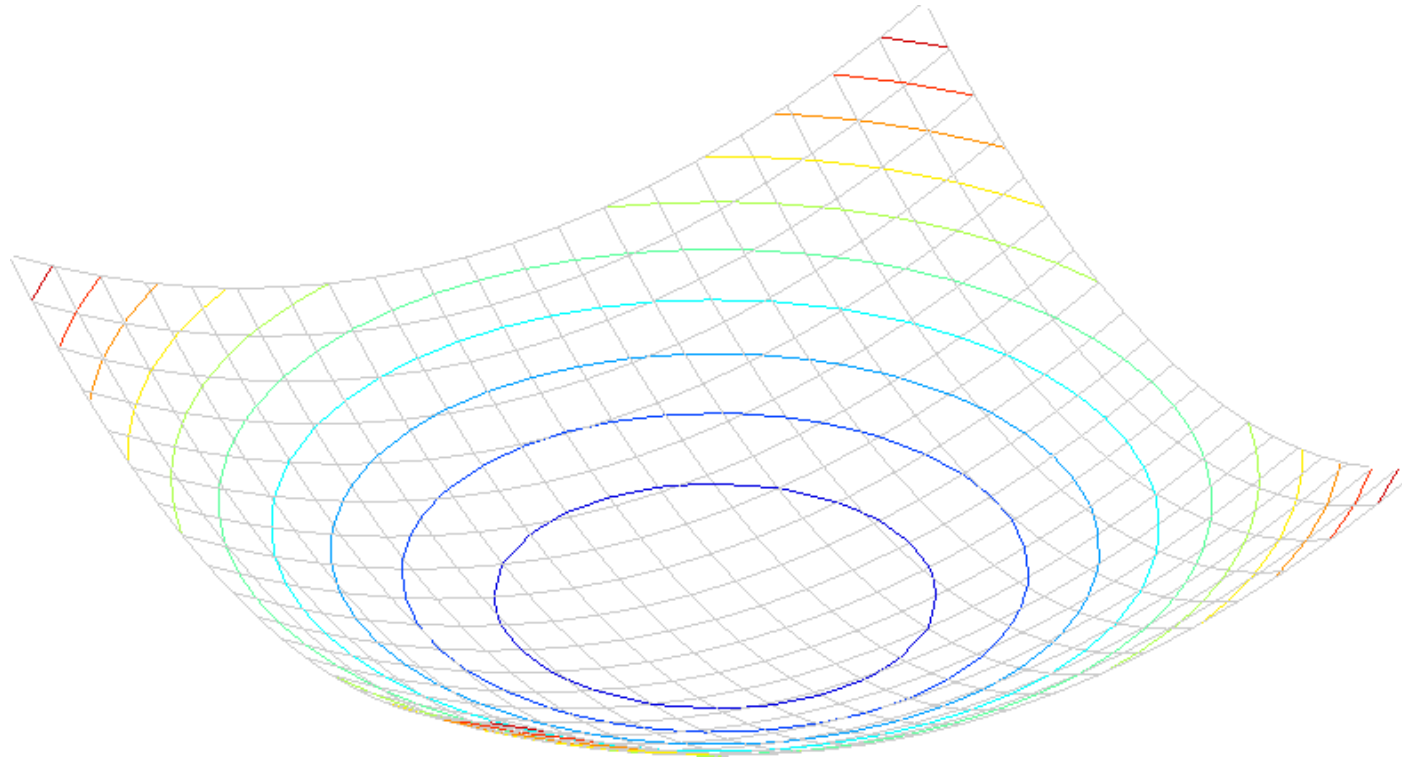
If either a or b is close to 0, then this is **not** a corner, so look for locations where both are large.

if a and b is large then it is a corner
but it only works for  not for 

Interpreting the Second Moment Matrix

Consider a horizontal “slice” of $E(u, v)$: $[u \ v] M \begin{bmatrix} u \\ v \end{bmatrix} = \text{const}$

This is the equation of an ellipse.



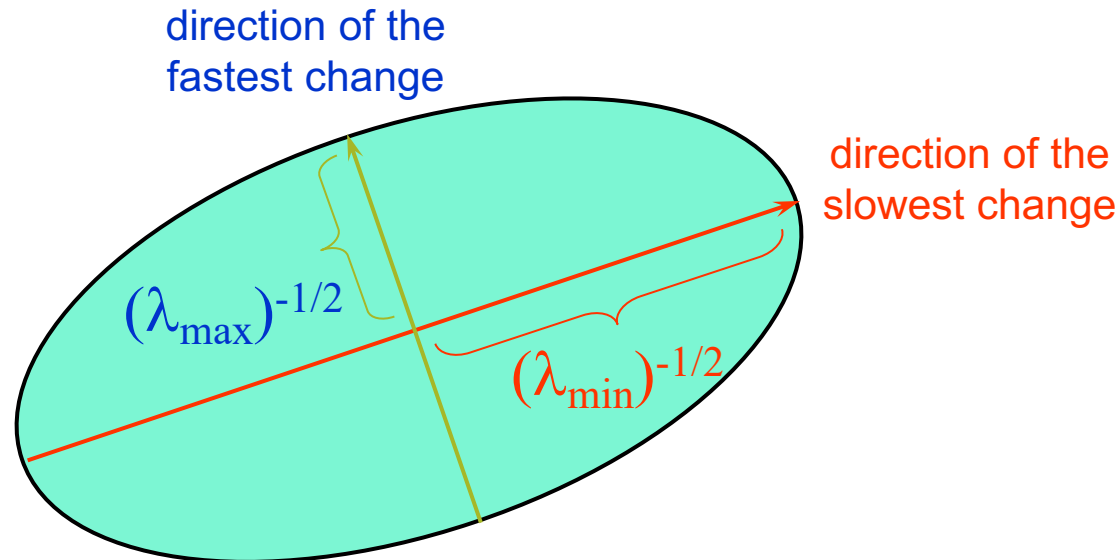
Interpreting the Second Moment Matrix

Consider a horizontal “slice” of $E(u, v)$: $[u \ v] M \begin{bmatrix} u \\ v \end{bmatrix} = \text{const}$

This is the equation of an ellipse.

Diagonalization of M : $M = R^{-1} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R$ *← Rotational matrix*

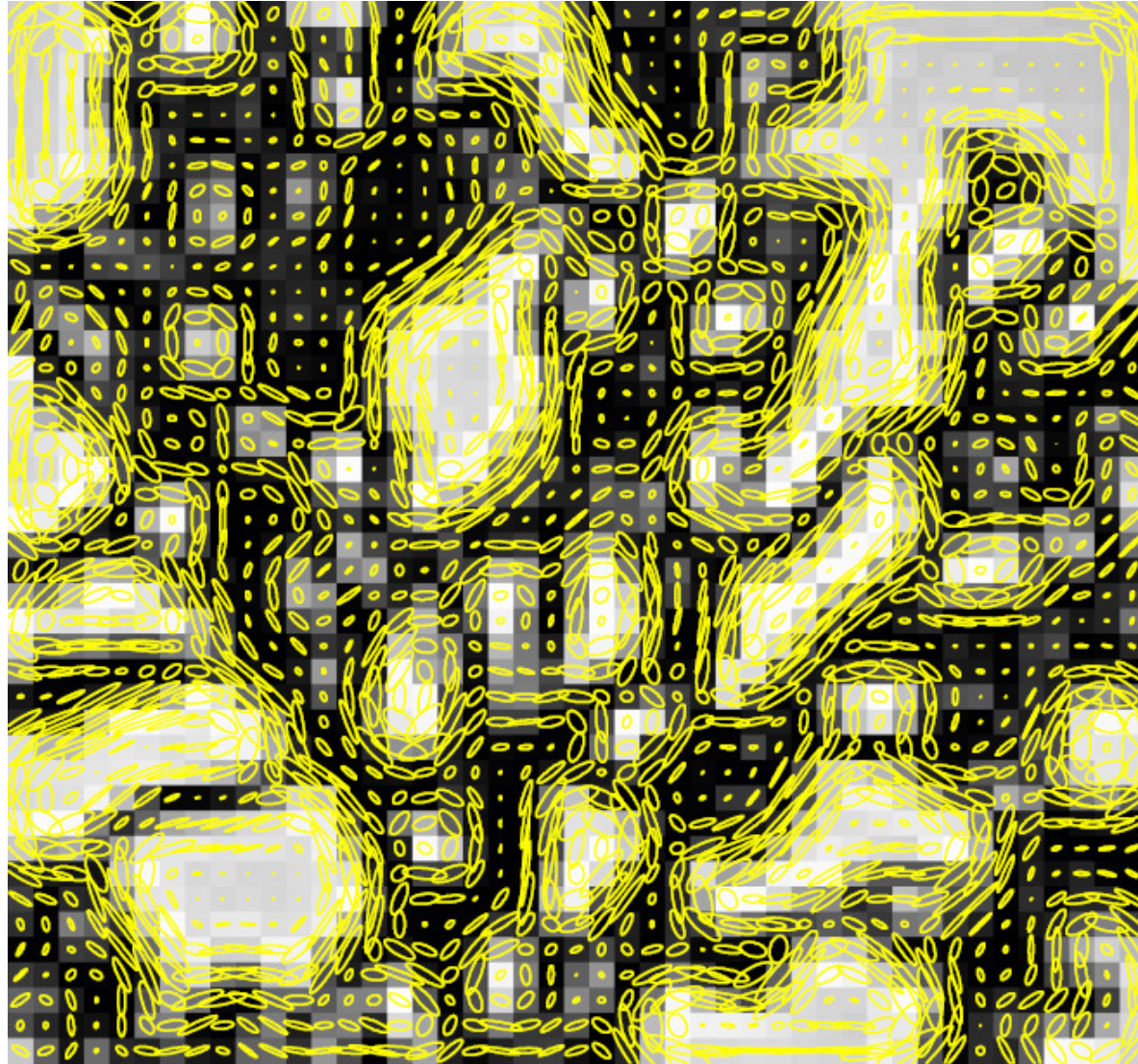
The axis lengths of the ellipse are determined by the eigenvalues and the orientation is determined by R



Visualization of Second Moment Matrices



Visualization of Second Moment Matrices



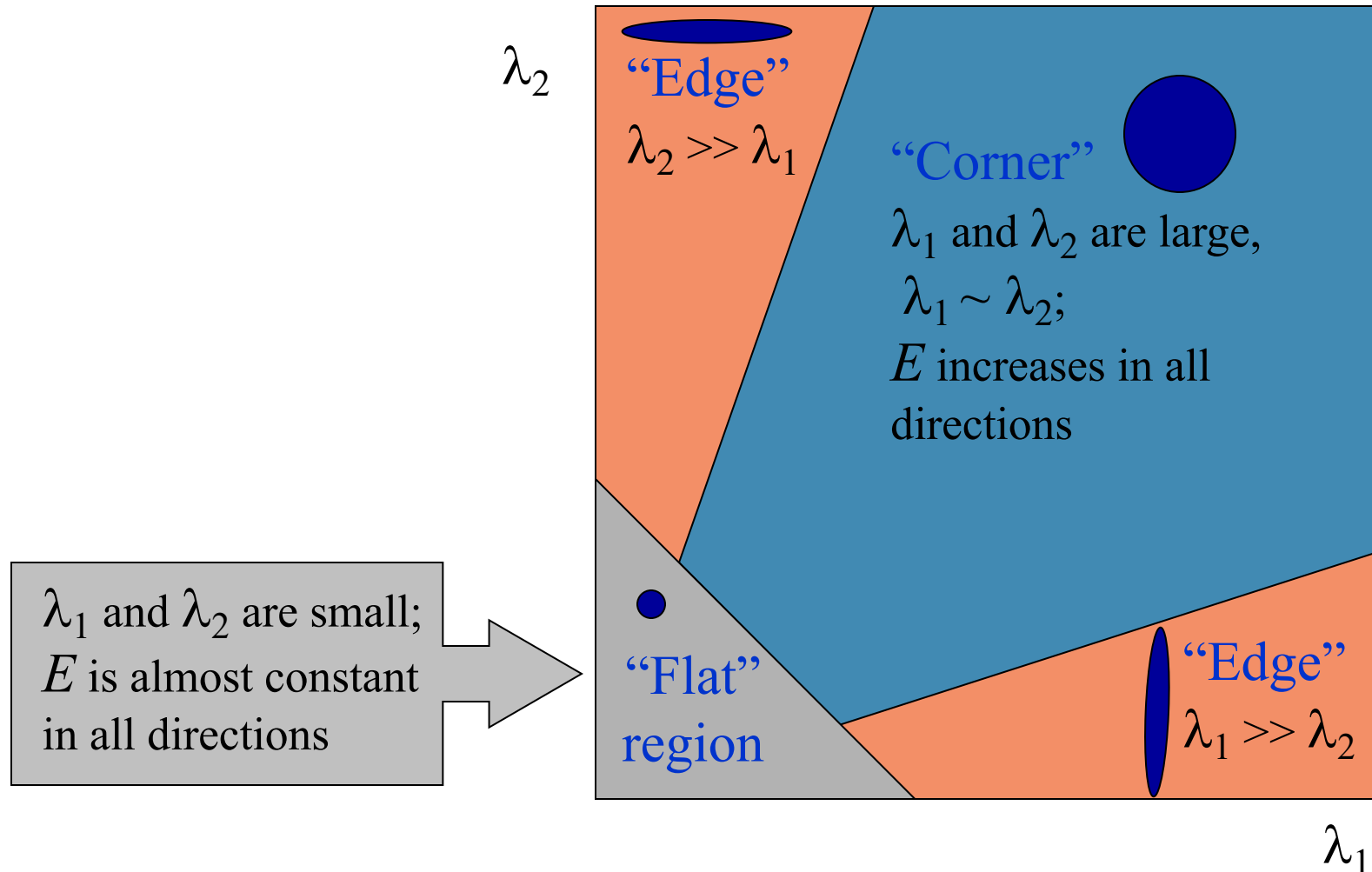
How to get m ?
do sobel x and y
on each pixel.
and multiply?

Interpreting the Eigenvalues

Classification of image points using eigenvalues of M :

*compute m
to each pixel*

*This is too
computationally
expensive*



Corner Response Function

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}_{2 \times 2}$$

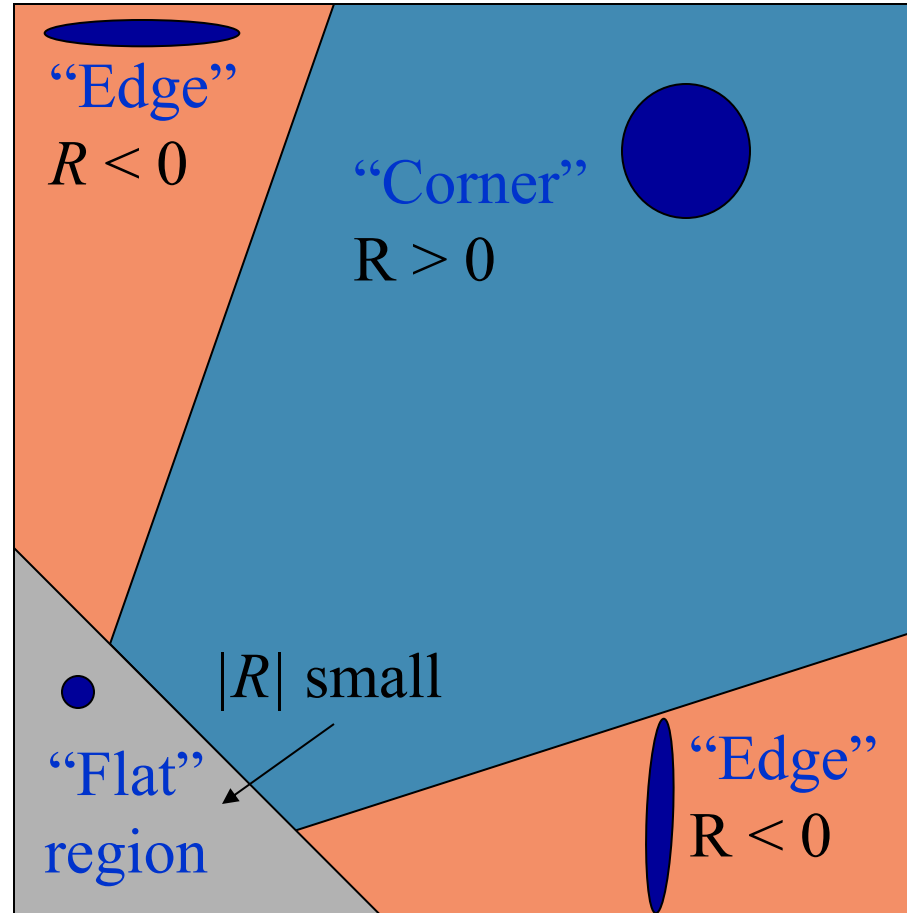
$\det(M) = ad - bc$
 $\text{trace}(M) = a + d$
easy to compute

$$R = \det(M) - \alpha \text{trace}(M)^2 = \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2$$

α : constant (0.04 to 0.06)

found experimentally

experimentally determined constant



The Harris Corner Detector

1. Compute partial derivatives at each pixel
2. Compute second moment matrix M in a Gaussian window around each pixel:

$$M = \begin{bmatrix} \sum_{x,y} w(x,y) I_x^2 & \sum_{x,y} w(x,y) I_x I_y \\ \sum_{x,y} w(x,y) I_x I_y & \sum_{x,y} w(x,y) I_y^2 \end{bmatrix}$$

} do sobel - x
sobel - y
get I_x I_y then
sum it.
due to 2nd power it
is not linear so we
cannot do grad of
Gaussian smth like before

C.Harris and M.Stephens. ["A Combined Corner and Edge Detector."](#)
Proceedings of the 4th Alvey Vision Conference: pages 147—151, 1988.

The Harris Corner Detector

1. Compute partial derivatives at each pixel
2. Compute second moment matrix M in a Gaussian window around each pixel
3. Compute corner response function R

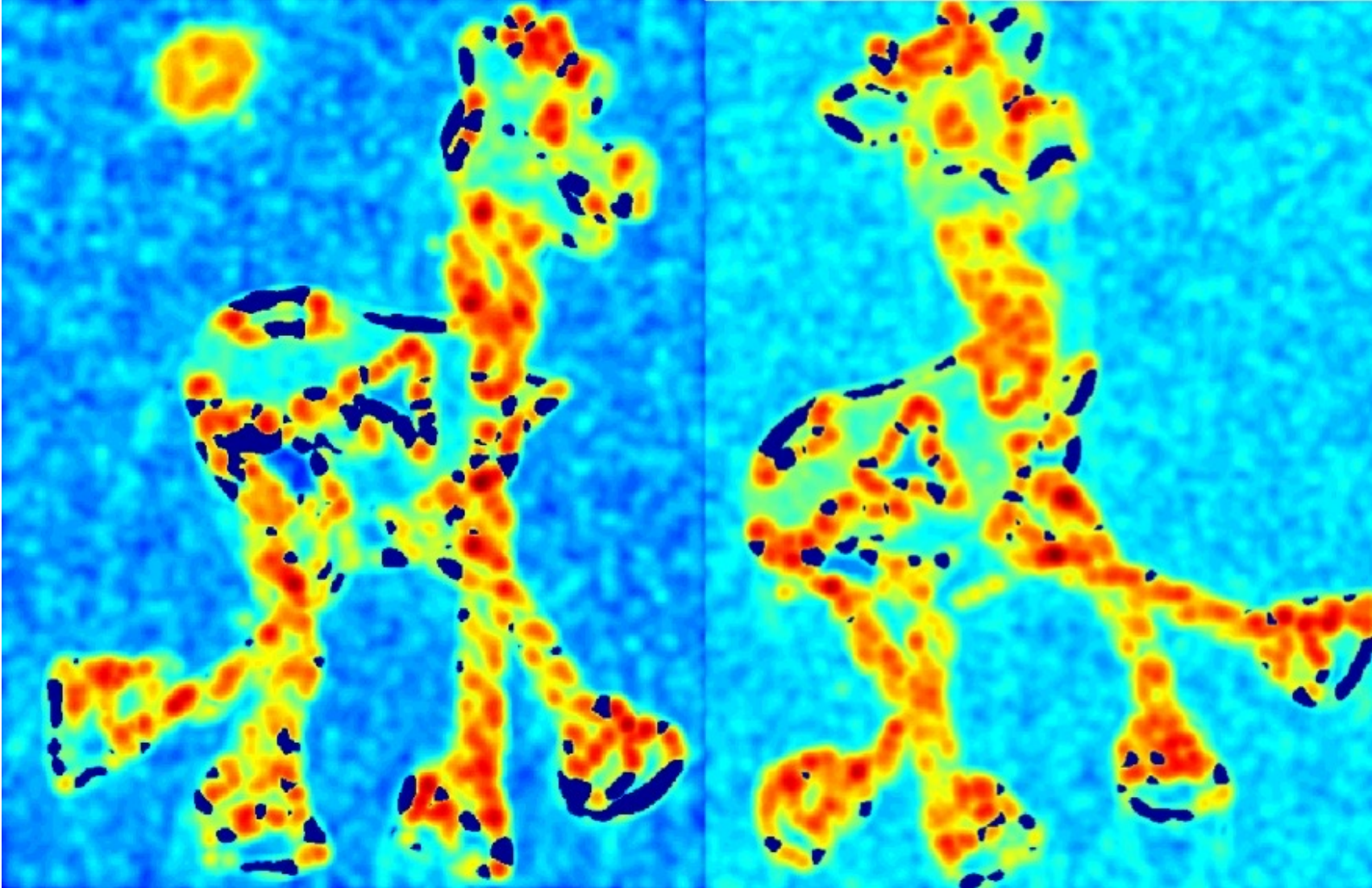
C.Harris and M.Stephens. [“A Combined Corner and Edge Detector.”](#)
Proceedings of the 4th Alvey Vision Conference: pages 147—151, 1988.

Harris Detector: Steps



Harris Detector: Steps

Compute corner response R



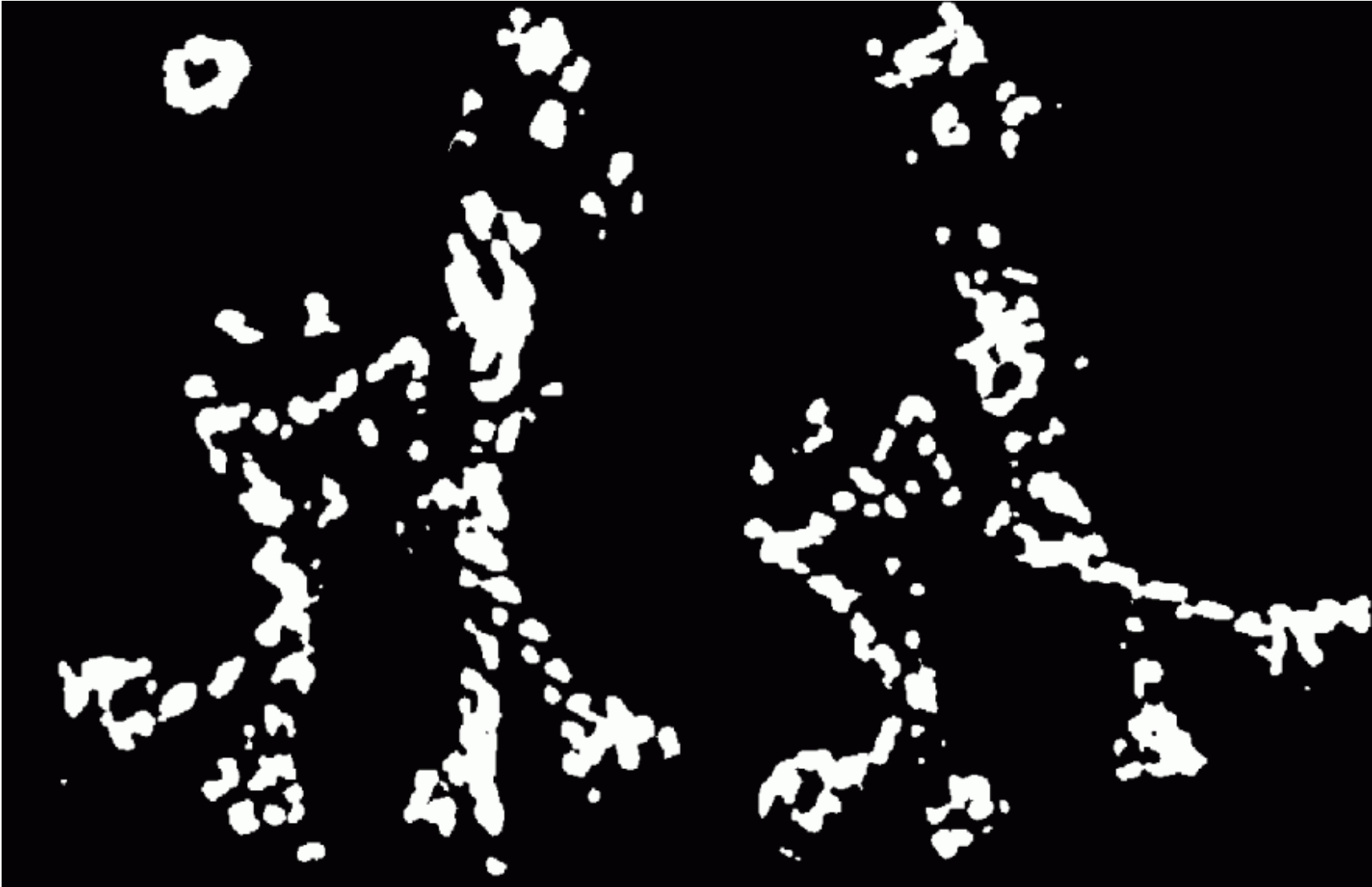
The Harris Corner Detector

1. Compute partial derivatives at each pixel
2. Compute second moment matrix M in a Gaussian window around each pixel
3. Compute corner response function R
4. Threshold R
5. Find local maxima of response function (nonmaximum suppression)

C.Harris and M.Stephens. ["A Combined Corner and Edge Detector."](#)
Proceedings of the 4th Alvey Vision Conference: pages 147—151, 1988.

Harris Detector: Steps

Find points with large corner response: $R > \text{threshold}$



Harris Detector: Steps

Take only the points of local maxima of R



Harris Detector: Steps



Invariance and Covariance

We want corner locations to be *invariant* to photometric transformations and *covariant* to geometric transformations

- **Invariance:** image is transformed and corner locations do not change
- **Covariance:** if we have two transformed versions of the same image, features should be detected in corresponding locations



lighting changes

rotations
scaling

Activity

Discuss in pairs and decide:

I may randomly ask a pair to respond.

Is Harris corner detection invariant to

1. Affine intensity changes $I \rightarrow aI + b$
2. Image translation
3. Image rotation
4. Image scaling

*$dZ = a dZ$ grad will change
so location will not change but some
corners may appear or disappear due to
 a .*

Affine Intensity Change

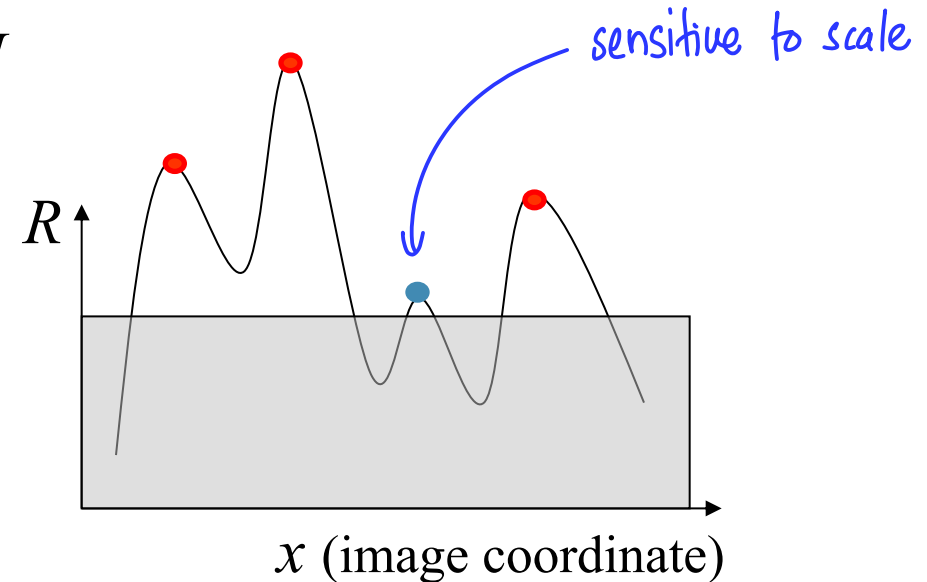
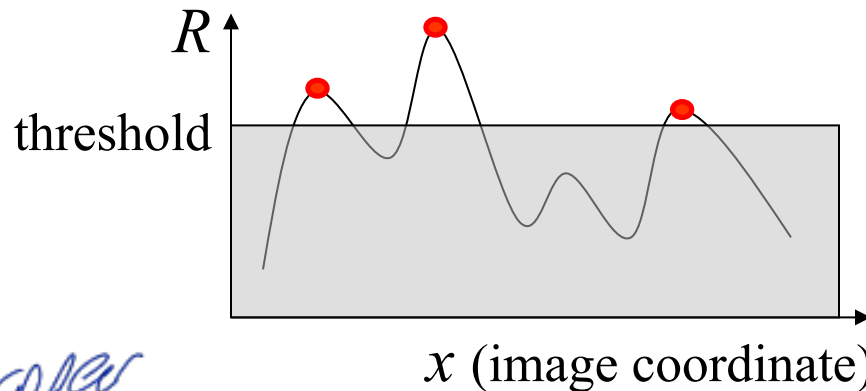


$$I \rightarrow aI + b$$

- Only derivatives are used =>
invariance to intensity shift $I \rightarrow I + b$

sensitive to scaling

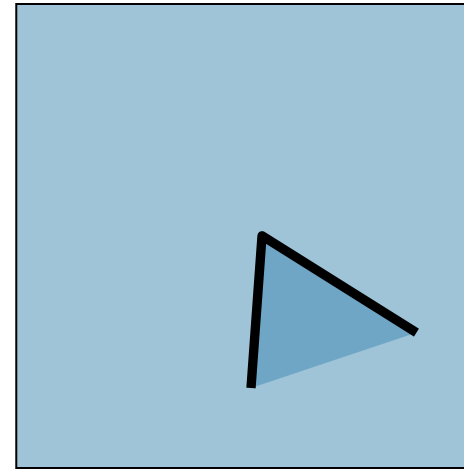
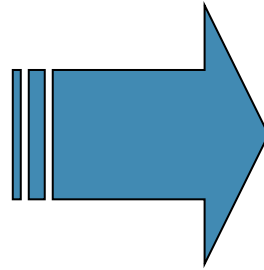
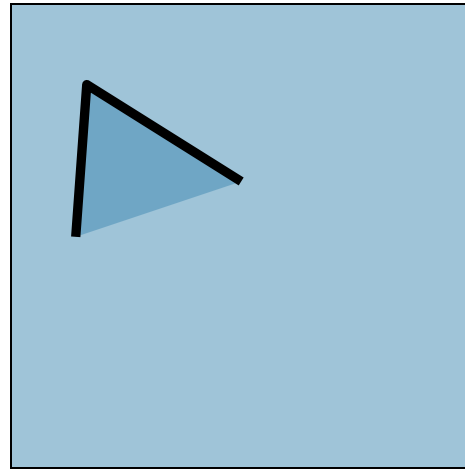
- Intensity scaling: $I \rightarrow aI$



*some may prefer
of dissimilarity*

Partially invariant to affine intensity change

Image Translation *(me)*



*Covariant
⇒
corners will
change name*

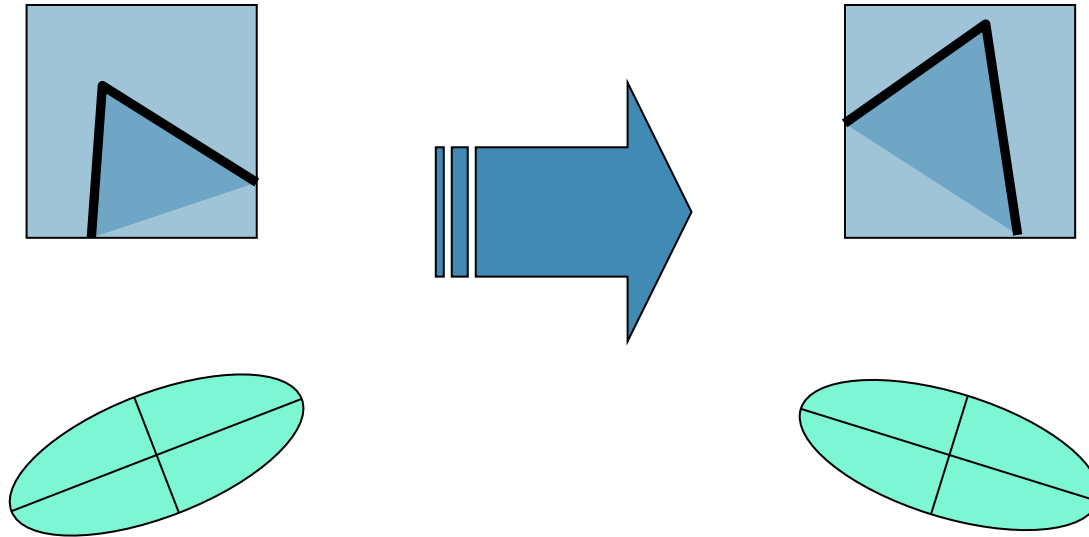
- Derivatives and window function are shift-invariant

Corner location is covariant w.r.t. translation

↪ not invariant

Image Rotation

* Quiz question

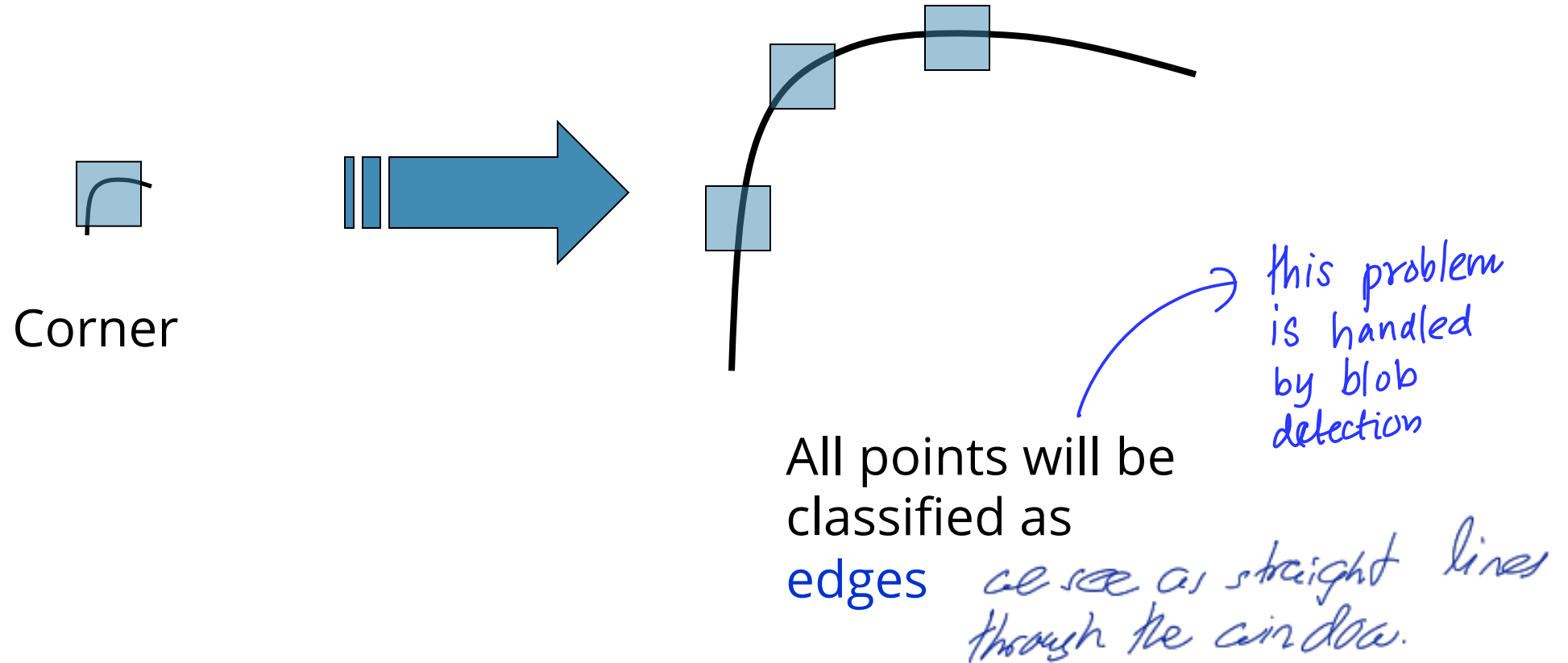


Second moment ellipse rotates but its shape (i.e. eigenvalues) remains the same

Corner location is covariant w.r.t. rotation

Scaling

image $\Rightarrow$ enlarged
window $\Rightarrow$ not enlarged



Corner location is not covariant to scaling!

Summary

Know that $\text{trace}(A) = h_1 + h_2$
 $\det(A) = h_1 h_2$

why $\Rightarrow$ Repeatability
Saliency

Compactness
locality

uses $\Rightarrow$ 3D-reconstruction
Object detection
motion tracking

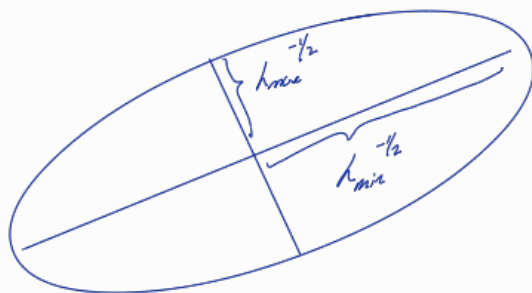
robot navigation
image alignment

$$E(u, v) = \sum_{x, y \in W} [I(x+u, y+v) - I(x, y)]^2$$
$$= \sum (I_x u + I_y v)^2$$

$$\begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

To get I_x what we do is $I * K_x \leftarrow$ Sobel x
and I_y will change for each pixel.

1 method find h_1 and h_2 of M
if both close to 0 $\Rightarrow$ flat
if $h_1 \gg h_2$ or $h_2 \gg h_1 \Rightarrow$ edge



if both h_1, h_2 large $\Rightarrow$ corner

2 method

$$R = \det(M) - \underset{\substack{\uparrow \\ [0.04 - 0.06]}}{2[\text{trace}(M)]}^2$$

$R \approx 0 \Rightarrow \text{Flat}$

$R < 0 \Rightarrow \text{Edge}$

$R > 0 \Rightarrow \text{corner}$

We want them to be invariant to photometric transform
covariant to geometric transform

- partially invariant to affine intensity change
- Covariant to translation and rotation
- not covariant to scaling.

scaled space extrema method



Blob Detection

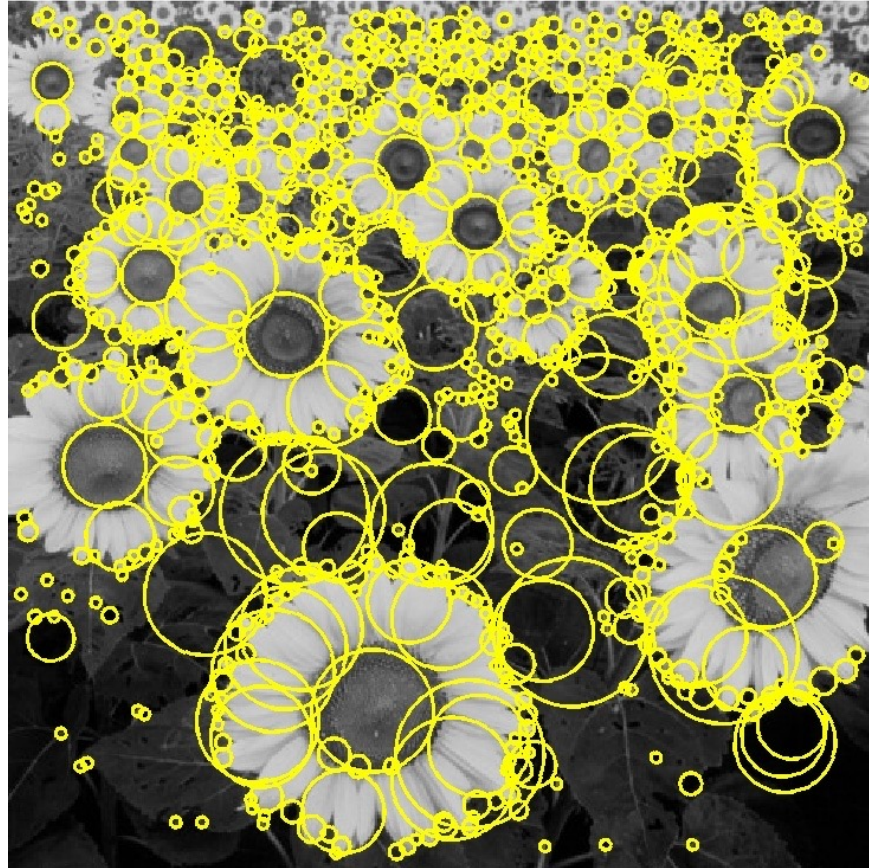
Slides from Svetlana Lazebnik

To get
scaled
invariance

To achieve scale invariance we need to know the
scale of the image.

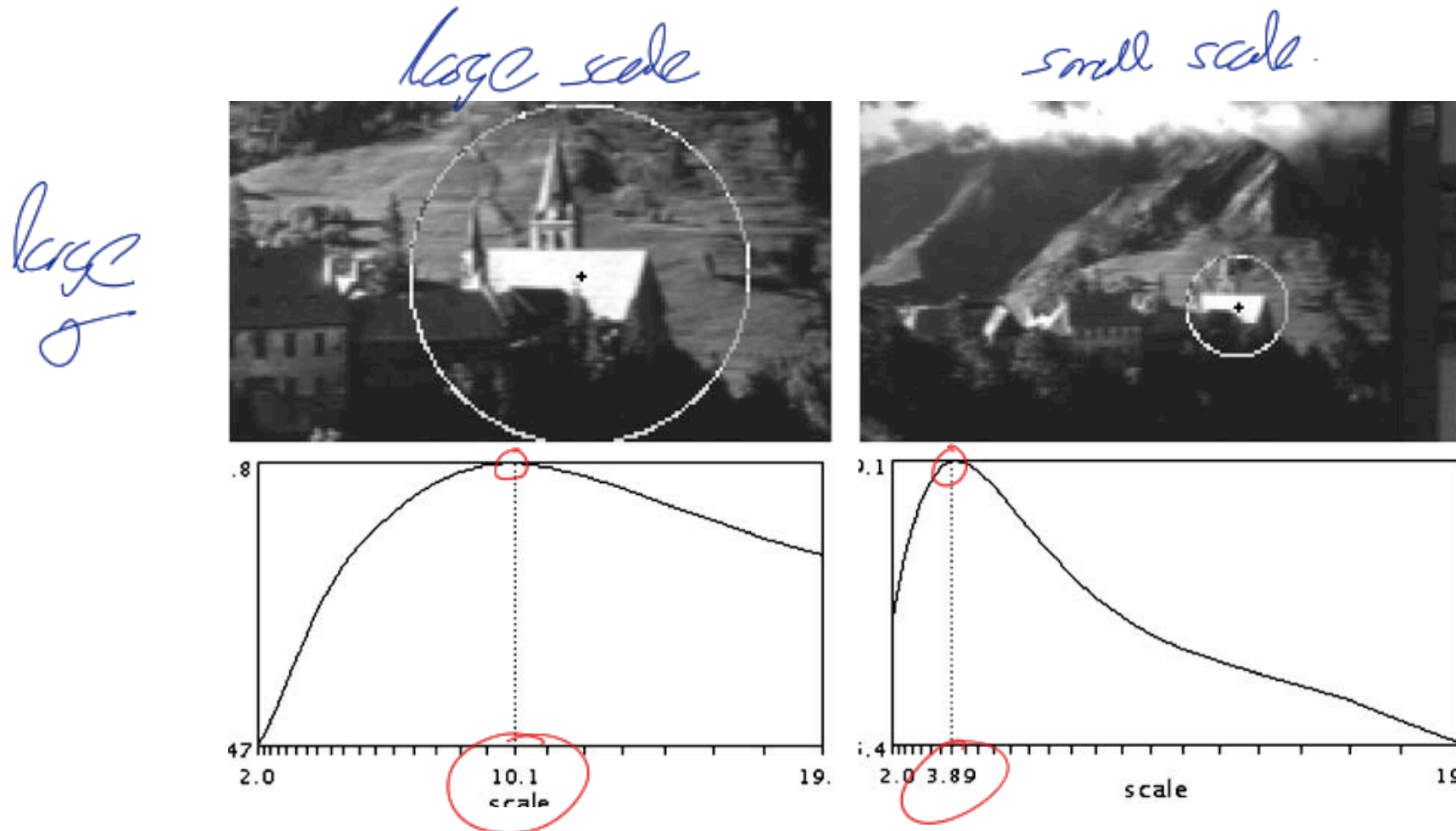
Blob Detection

① Different size of blobs



Feature Detection with Scale Selection

We want to extract features with characteristic scale that is *covariant* with the image transformation



We design the function to be maximum at the correct scale

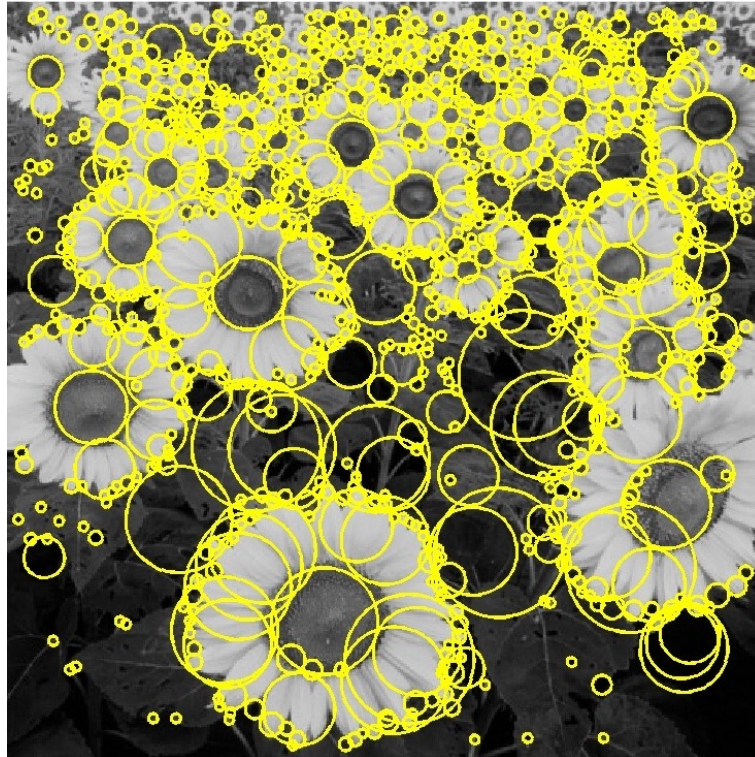
small

If we find this func and get the max, we can get the scale

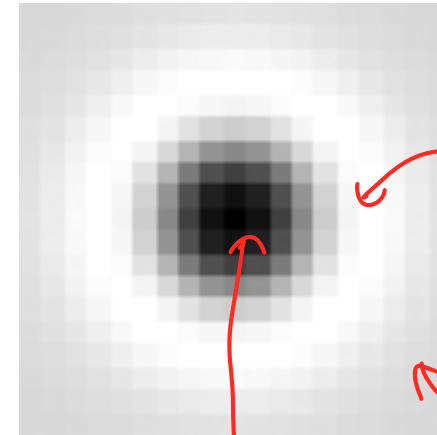
Blob Detection: Basic idea

To detect blobs, convolve the image with a “blob filter” at multiple scales and look for extrema of filter response in the resulting *scale space*.

*scale space
extrema detection*



blob filter



bright ring

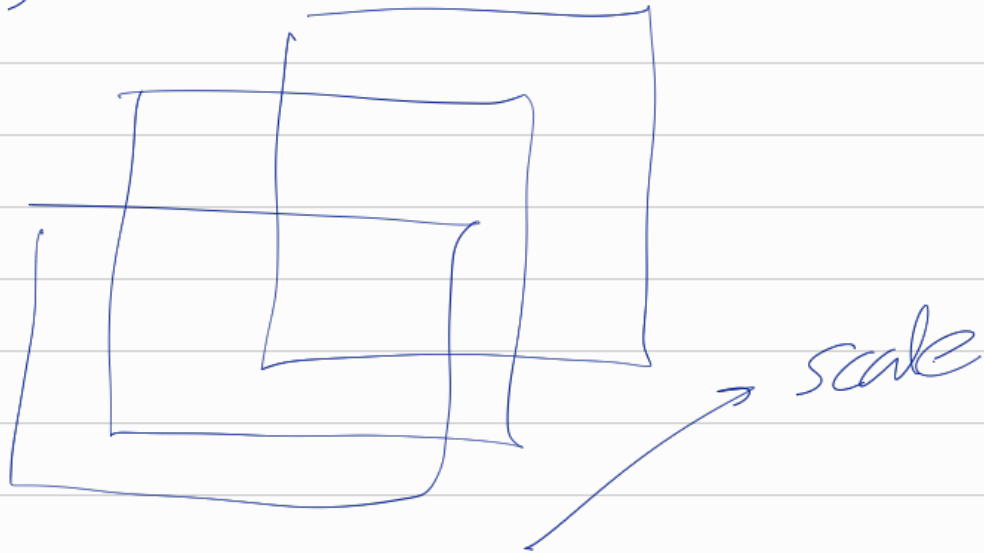
little dark

dark

Run the filter with multiple σ over the image

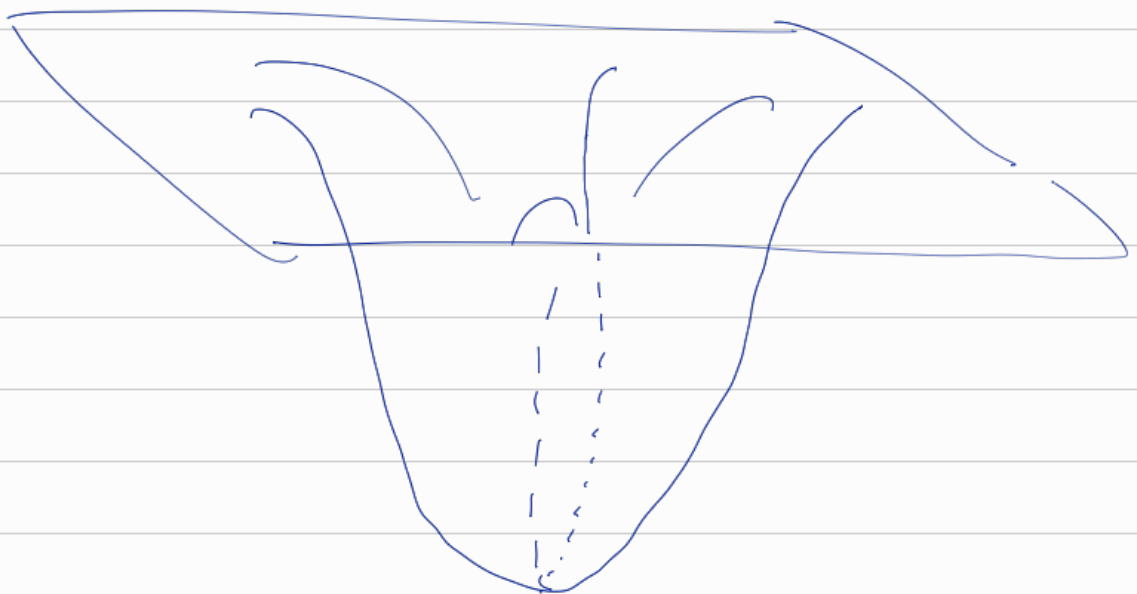
then we get several responses

By, scale

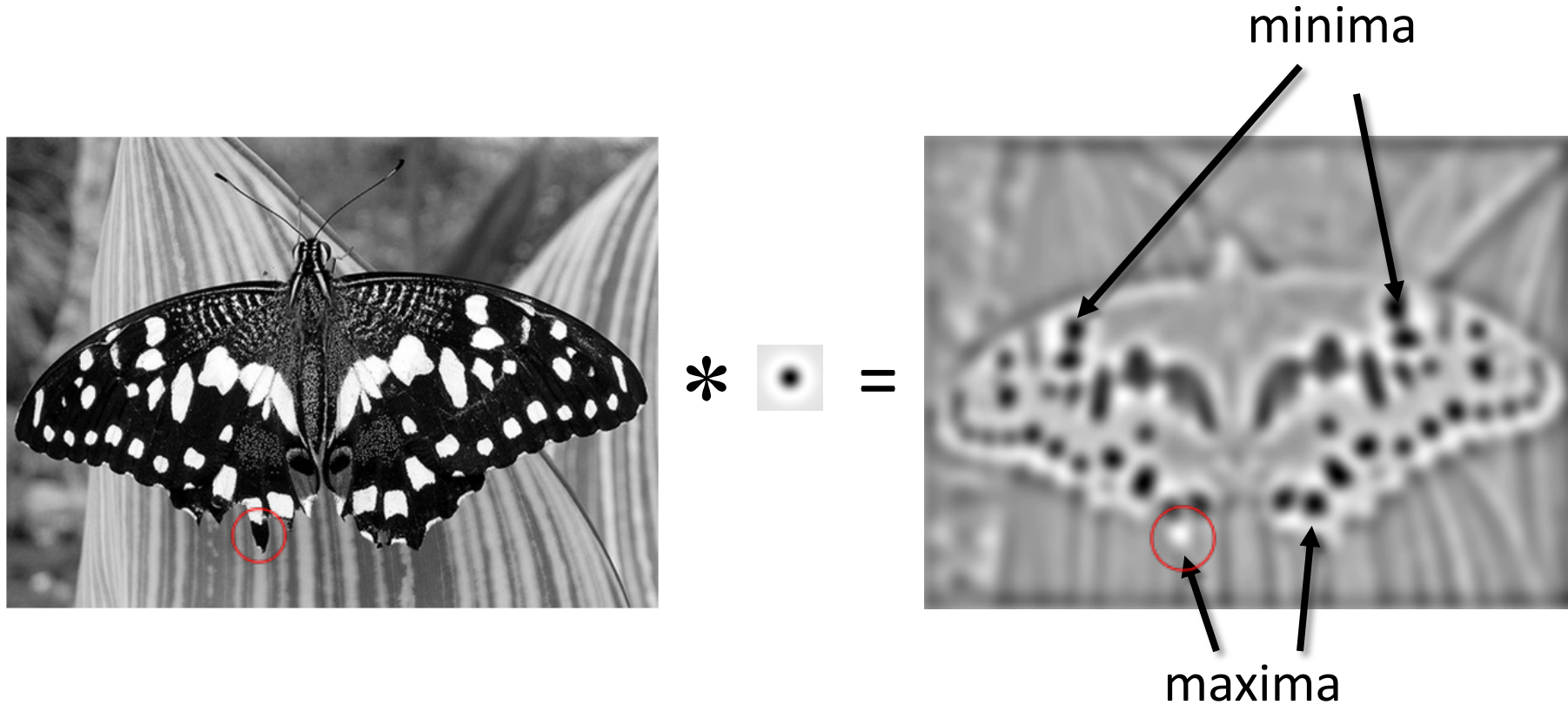


Large σ respond to large stuff

blob filter looks like - defined by σ



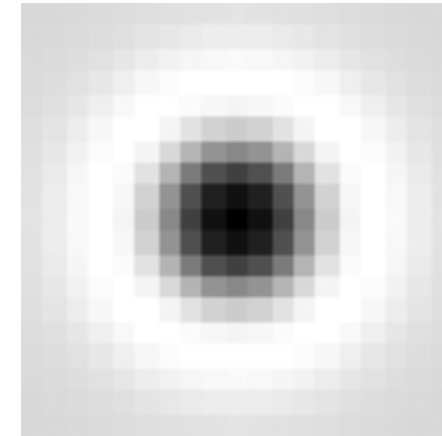
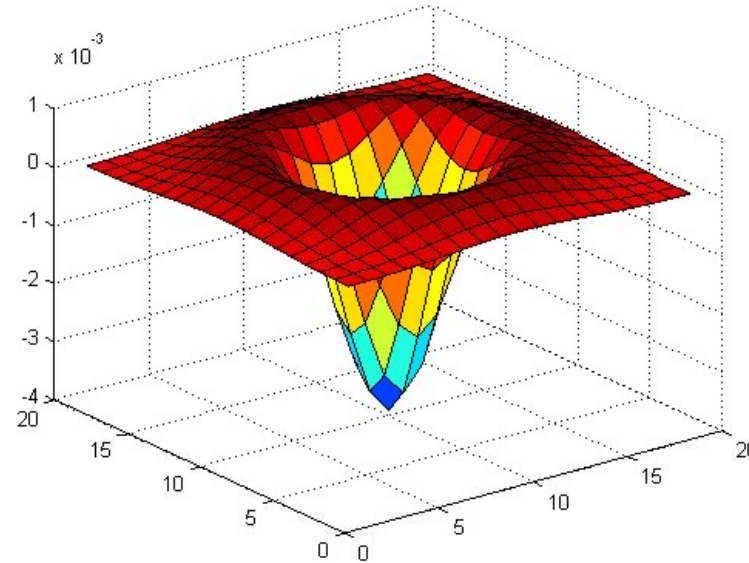
Blob Detection: Basic Idea



Find maxima *and minima* of blob filter response in space *and scale*

Blob Filter *is the laplacian of the gaussian*

Laplacian of Gaussian: Circularly symmetric operator for blob detection in 2D

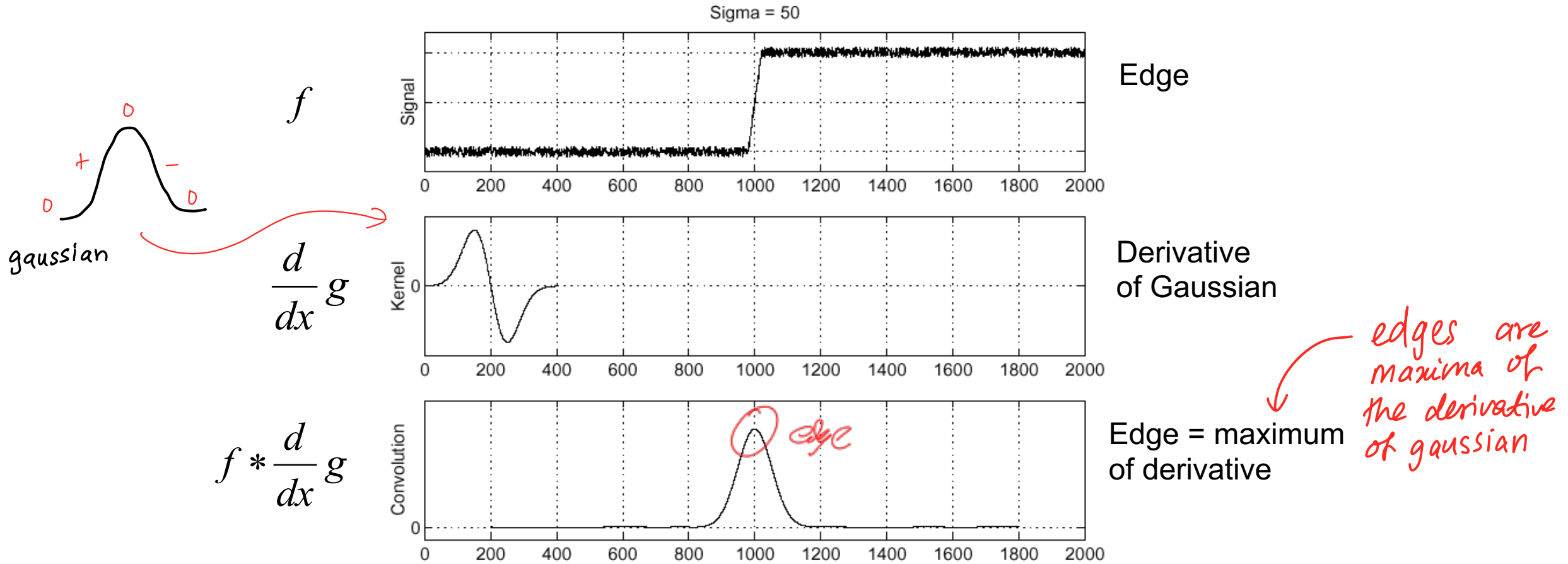


*middle dark
outer light*

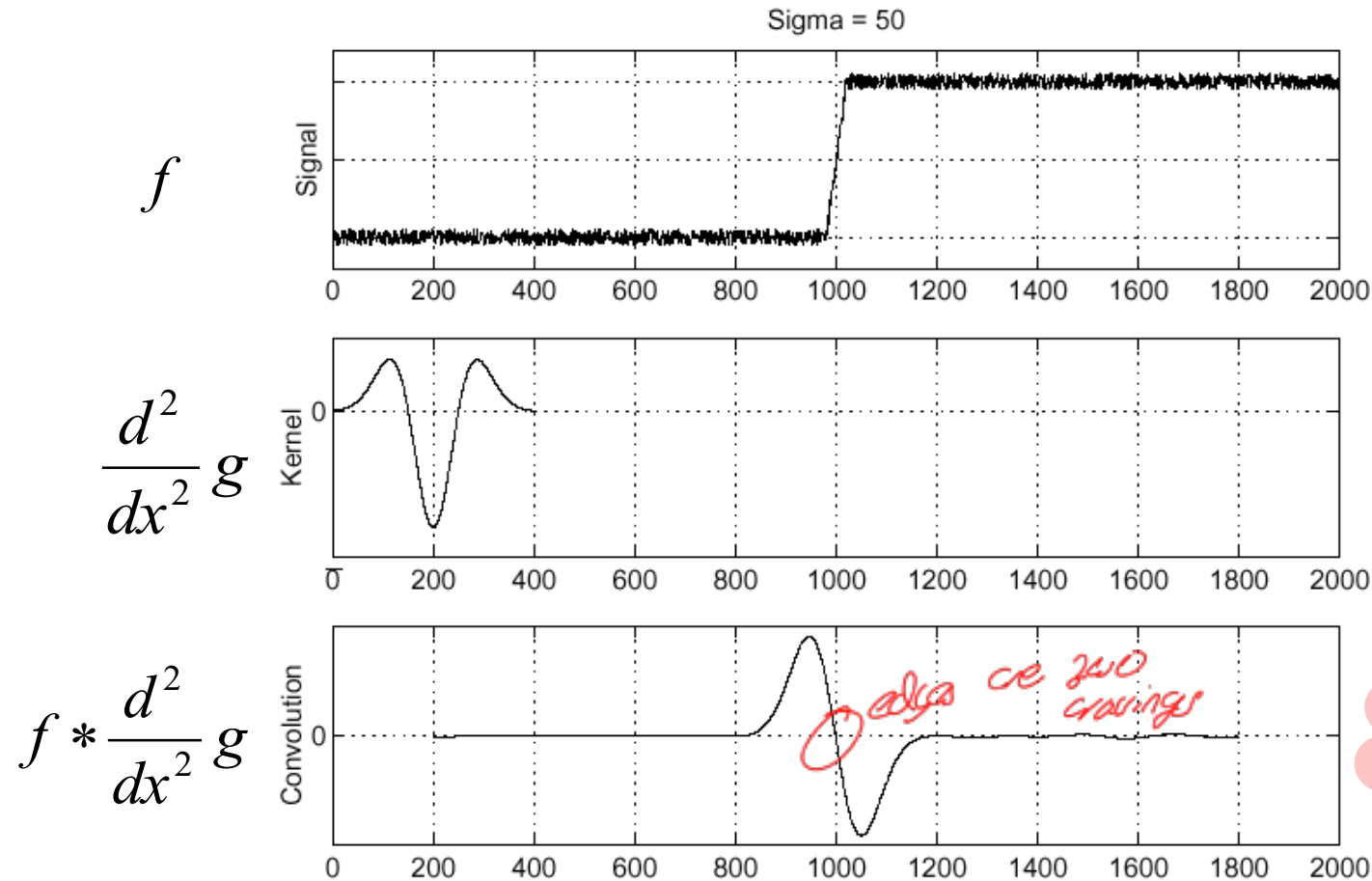
$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$$

second derivative

Recall: Edge Detection



Edge Detection, Take 2



Edge

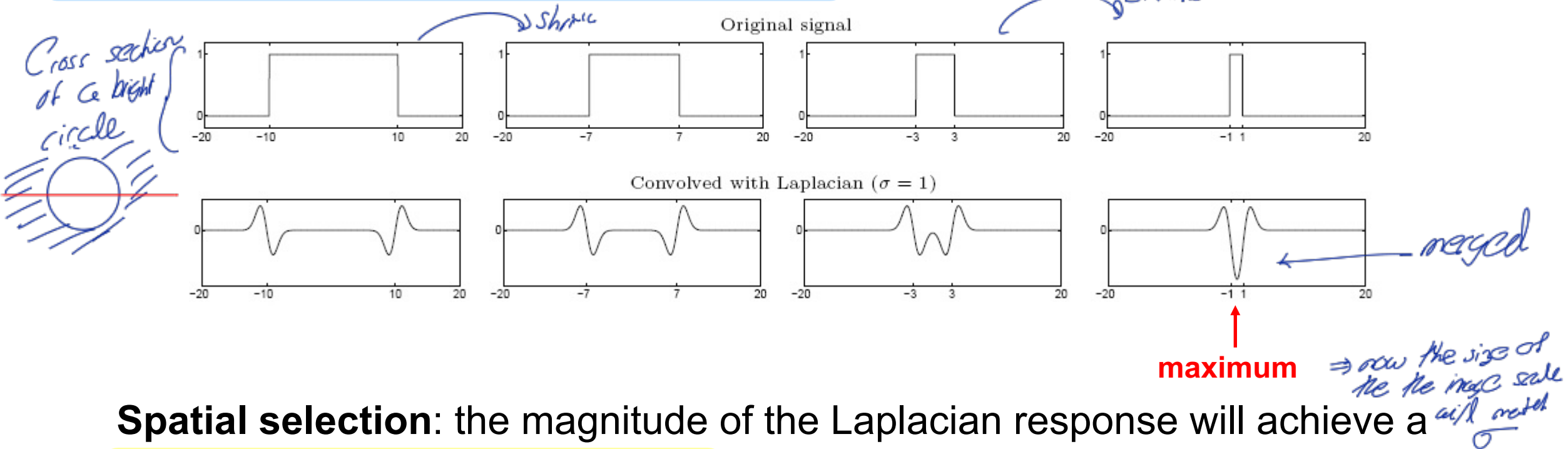
Second derivative
of Gaussian
(Laplacian)

Edge = zero crossing
of second derivative

From Edges to Blobs

- Edge = ripple
- Blob = superposition of two ripples

response is maximum when,
scale of laplacian = scale of the blob

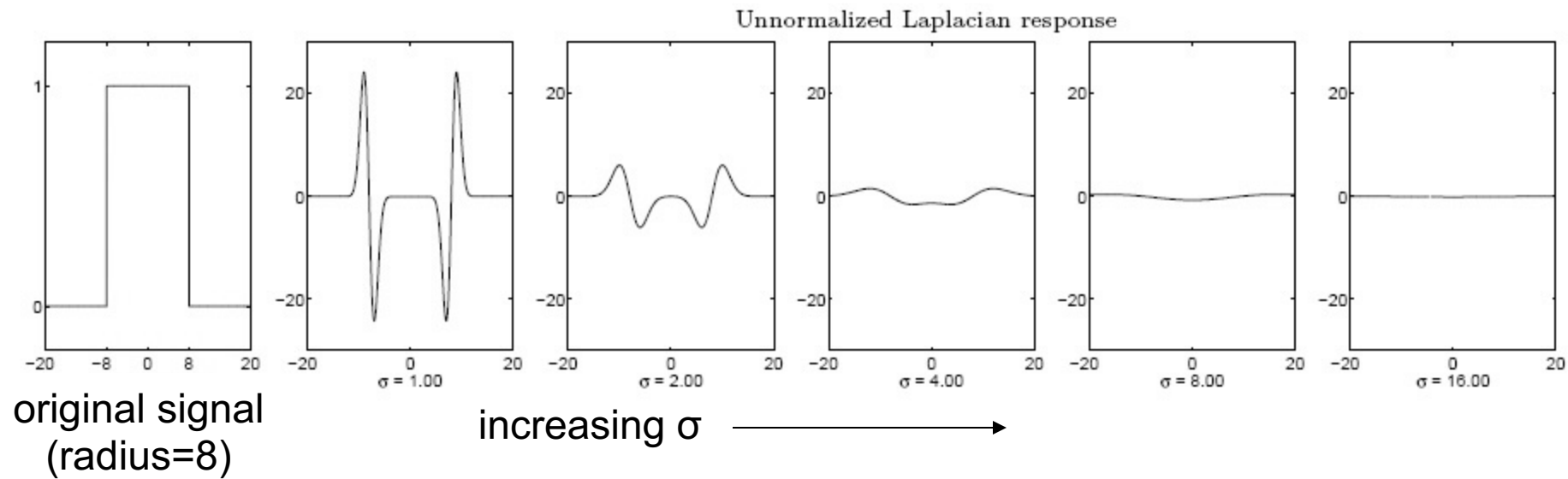


Spatial selection: the magnitude of the Laplacian response will achieve a maximum at the center of the blob, provided the scale of the Laplacian is “matched” to the scale of the blob

Very important

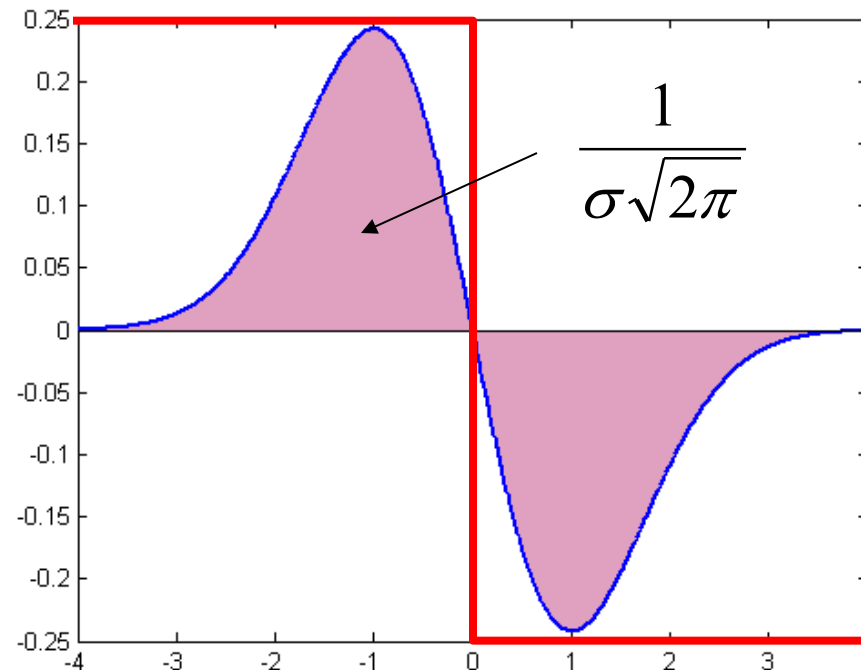
Scale Selection

- We want to find the characteristic scale of the blob by convolving it with Laplacians at several scales and looking for the maximum response
- However, Laplacian response decays as scale increases:



Scale Normalization

The response of a derivative of Gaussian filter to a perfect step edge decreases as σ increases



$\sigma \uparrow$ response $\downarrow$

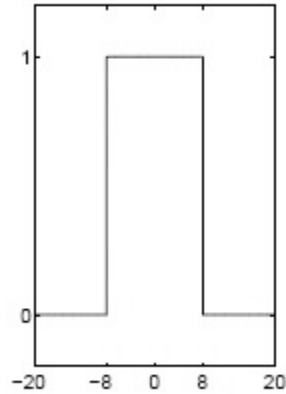
Scale Normalization

- The response of a derivative of Gaussian filter to a perfect step edge decreases as σ increases
- To keep response the same (scale-invariant), must multiply Gaussian derivative by σ
- Laplacian is the second Gaussian derivative, so it must be multiplied by σ^2

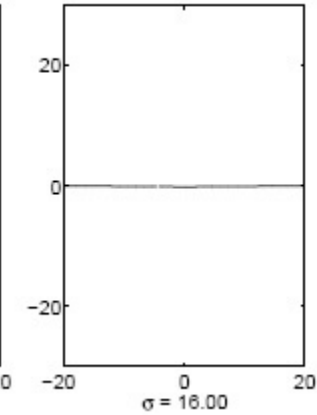
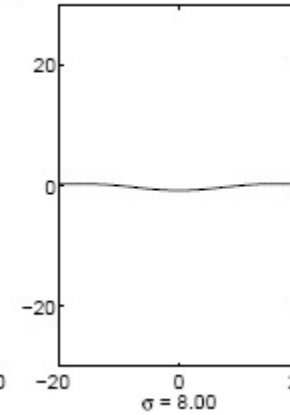
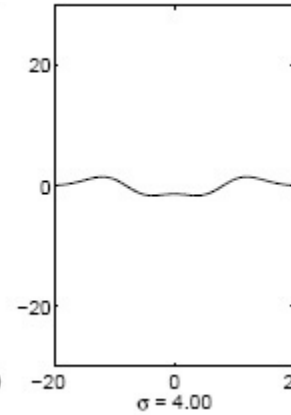
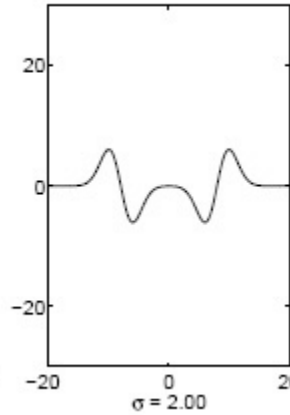
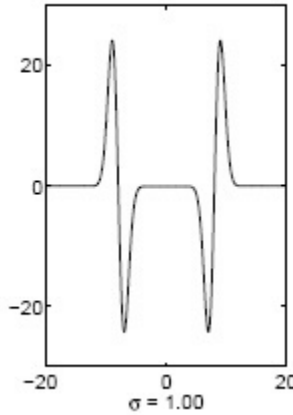
↑
Scale normalization

Effect of Scale Normalization

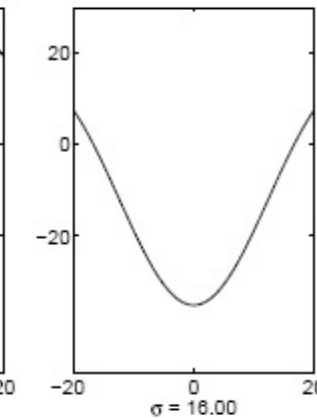
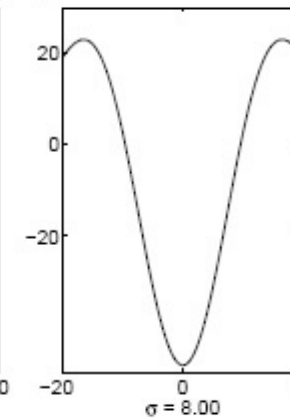
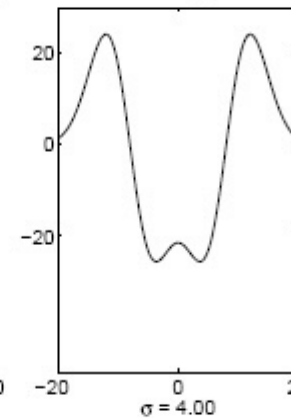
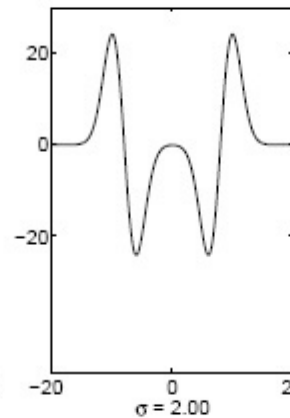
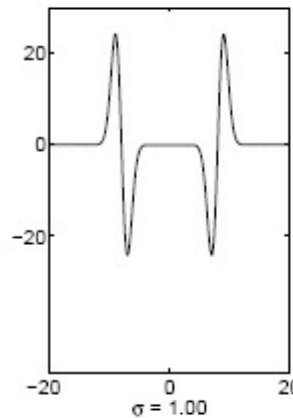
Original signal



Unnormalized Laplacian response



Scale-normalized Laplacian response



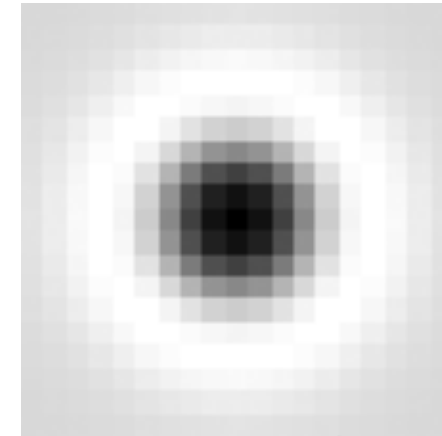
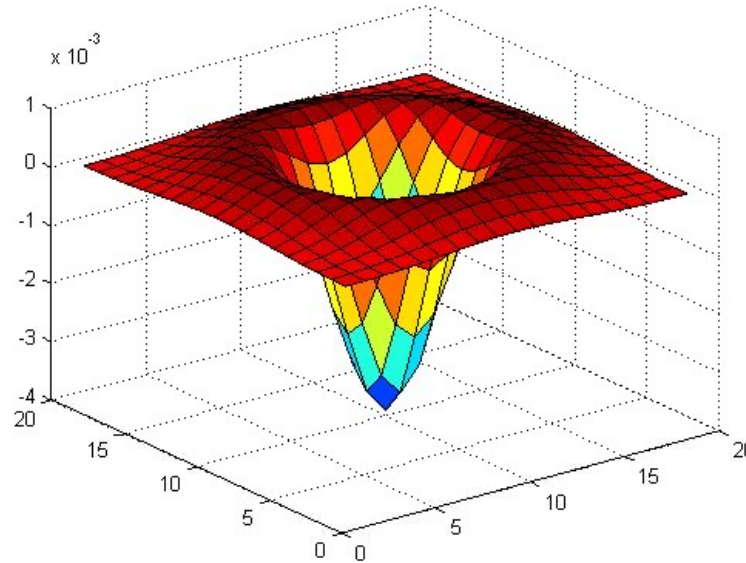
Scale
normalized



maximum (extremum)

Blob Detection in 2D

Laplacian of Gaussian: Circularly symmetric operator for blob detection in 2D



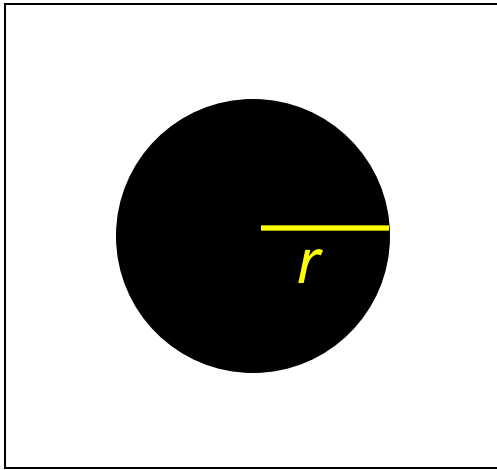
scale normalized (to remove the effect of σ & rescale)

Scale-normalized:

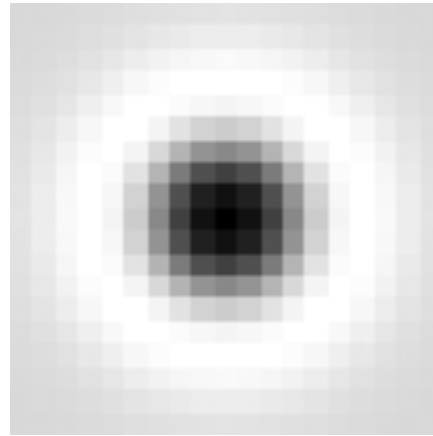
$$\nabla_{\text{norm}}^2 g = \sigma^2 \left(\frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2} \right)$$

Scale Selection

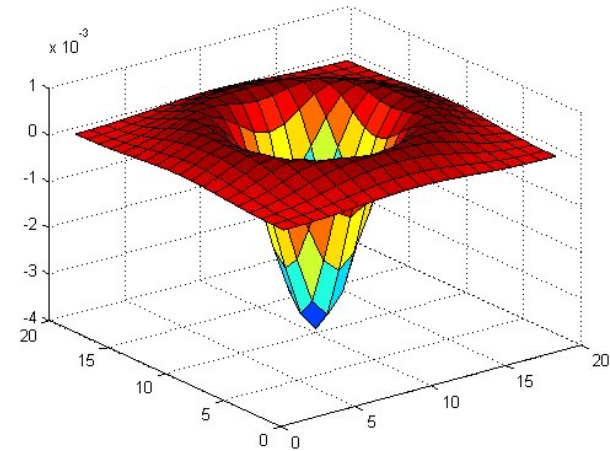
At what scale does the Laplacian achieve a maximum response to a binary circle of radius r ?



image



Laplacian

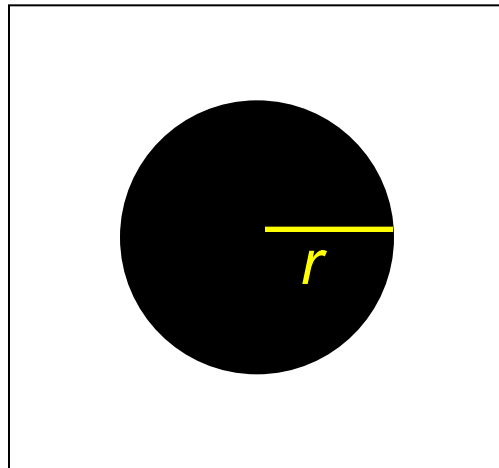


Scale Selection

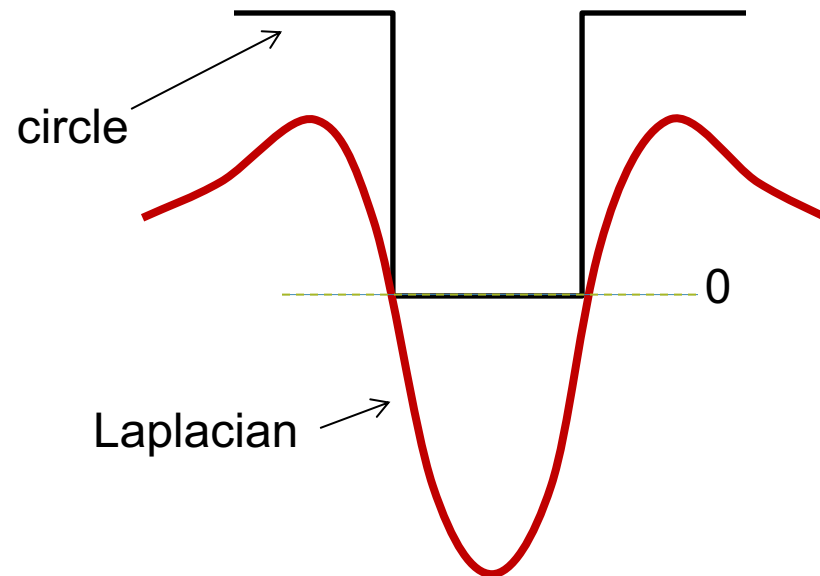
- At what scale does the Laplacian achieve a maximum response to a binary circle of radius r ?
- To get maximum response, the zeros of the Laplacian have to be aligned with the circle
- The Laplacian is given by (up to scale):
$$\textcircled{A} = (x^2 + y^2 - 2\sigma^2) e^{-(x^2 + y^2)/2\sigma^2}$$
- Therefore, the maximum response occurs at

$$\begin{aligned} x^2 + y^2 &= r^2 \\ r^2 - 2\sigma^2 &= 0 \end{aligned}$$

$$\sigma = r / \sqrt{2}.$$



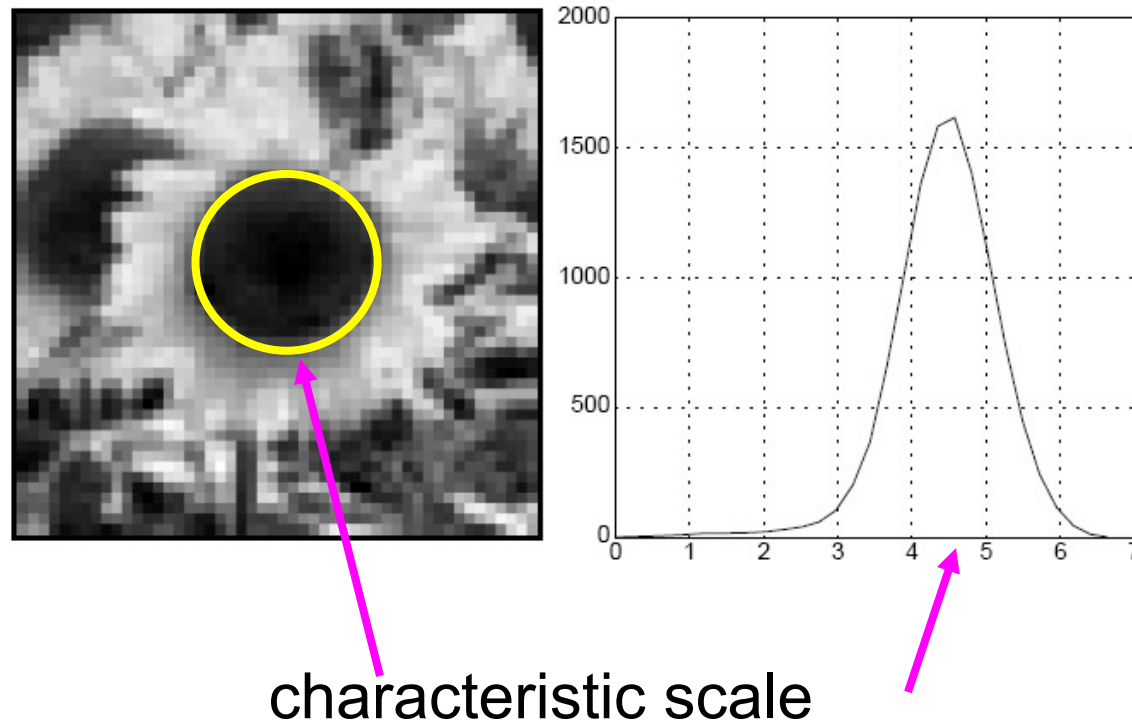
image



Circle the
Laplacian to
0 these are
edges

Characteristic Scale

We define the characteristic scale of a blob as the scale that produces peak of Laplacian response in the blob center



T. Lindeberg (1998). ["Feature detection with automatic scale selection."](#) *International Journal of Computer Vision* **30** (2): pp 77--116.

$$g(x, y) = \frac{1}{2\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

$$\frac{\partial g}{\partial x} = \frac{1}{2\sigma^2} e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)} \cdot \frac{-1}{2\sigma^2} \times 2x$$

$$\frac{\partial g}{\partial y} = \frac{1}{2\sigma^2} e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)} \times \frac{-1}{2\sigma^2} \times 2y$$

$$\begin{aligned} \frac{\partial^2 g}{\partial x^2} &= \frac{-1}{2\sigma^4} \left[e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)} + x e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)} \cdot \frac{-1}{2\sigma^2} \cdot 2x \right] \\ &= \frac{-1}{2\sigma^4} e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)} \left[1 - \frac{x^2}{\sigma^2} \right] \end{aligned}$$

$$\frac{\partial^2 g}{\partial y^2} = \frac{-1}{2\sigma^4} e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)} \left[1 - \frac{y^2}{\sigma^2} \right]$$

$$\begin{aligned} \sigma^2 \left[\frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2} \right] &= \frac{-1}{2\sigma^4} e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)} (2\sigma^2 - x^2 - y^2) \\ &= \frac{1}{2\sigma^4} e^{-\left(\frac{x^2+y^2}{2\sigma^2}\right)} (x^2 + y^2 - 2\sigma^2) \end{aligned}$$

$$\textcircled{1} + \textcircled{2} = \textcircled{A}$$

then in order to $\textcircled{A} = 0$

$$x^2 + y^2 = 2\sigma^2$$

Scale-Space Blob Detector

1. Convolve image with scale-normalized Laplacian at several scales

Scale-Space Blob Detector: Example



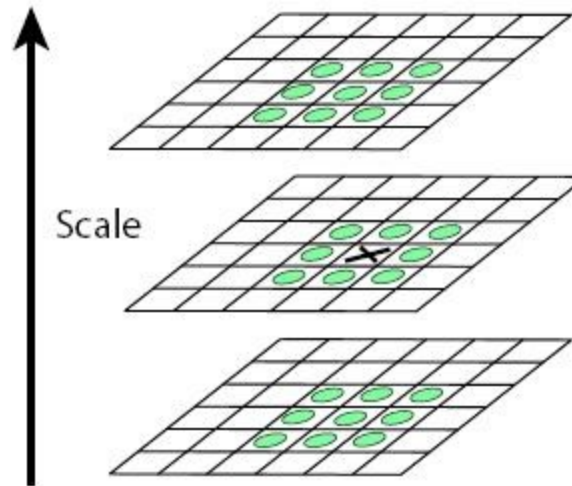
Scale-Space Blob Detector: Example



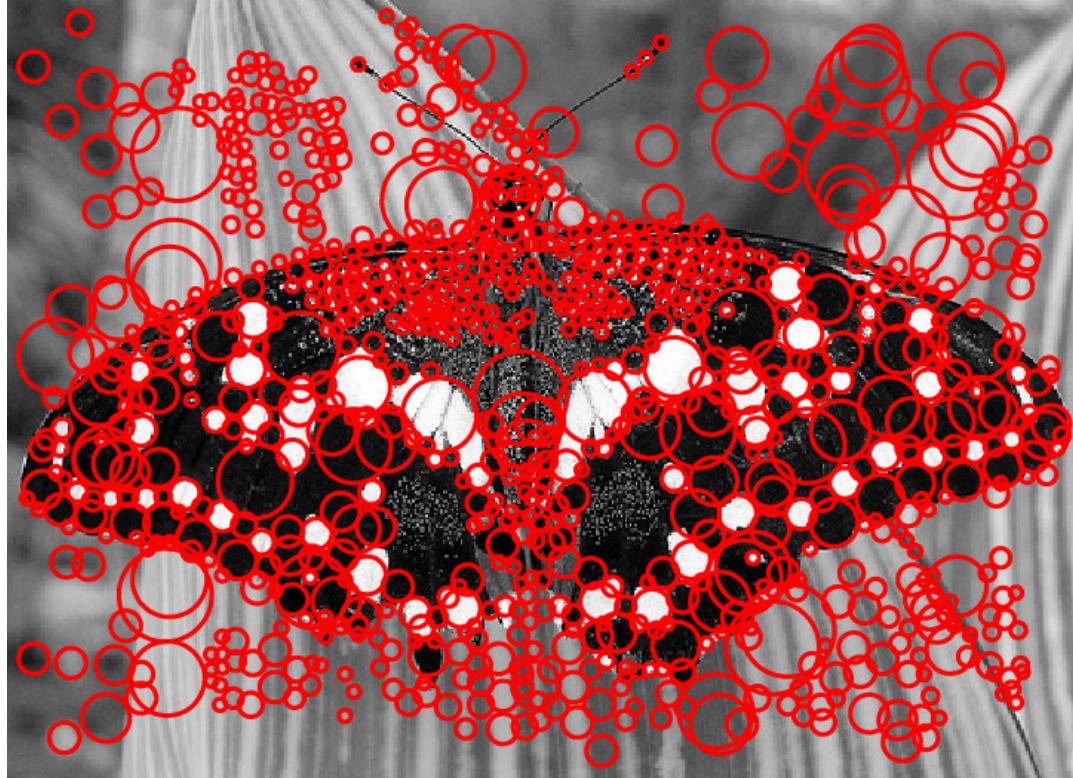
sigma = 11.9912

Scale-Space Blob Detector

1. Convolve image with scale-normalized Laplacian at several scales
2. Find maxima of squared Laplacian response in scale-space



Scale-Space Blob Detector: Example



Coin Detection Problem.

Assume that we are given an image containing more than one each of quarters, dimes, nickels, and pennies. Also, you are given template images of these coins in a standard 300x300 canvas.

Your task is to detect each coin and tell me the amount of money represented in the image.

Then, tell me what the assumptions are.



Efficient Implementation

Approximating the Laplacian with a difference of Gaussians:

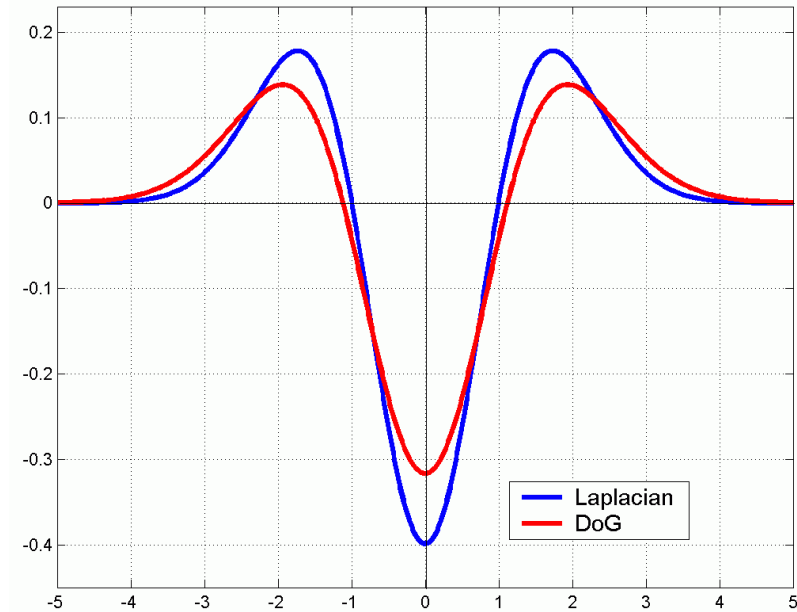
$$L = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right)$$

(Laplacian)

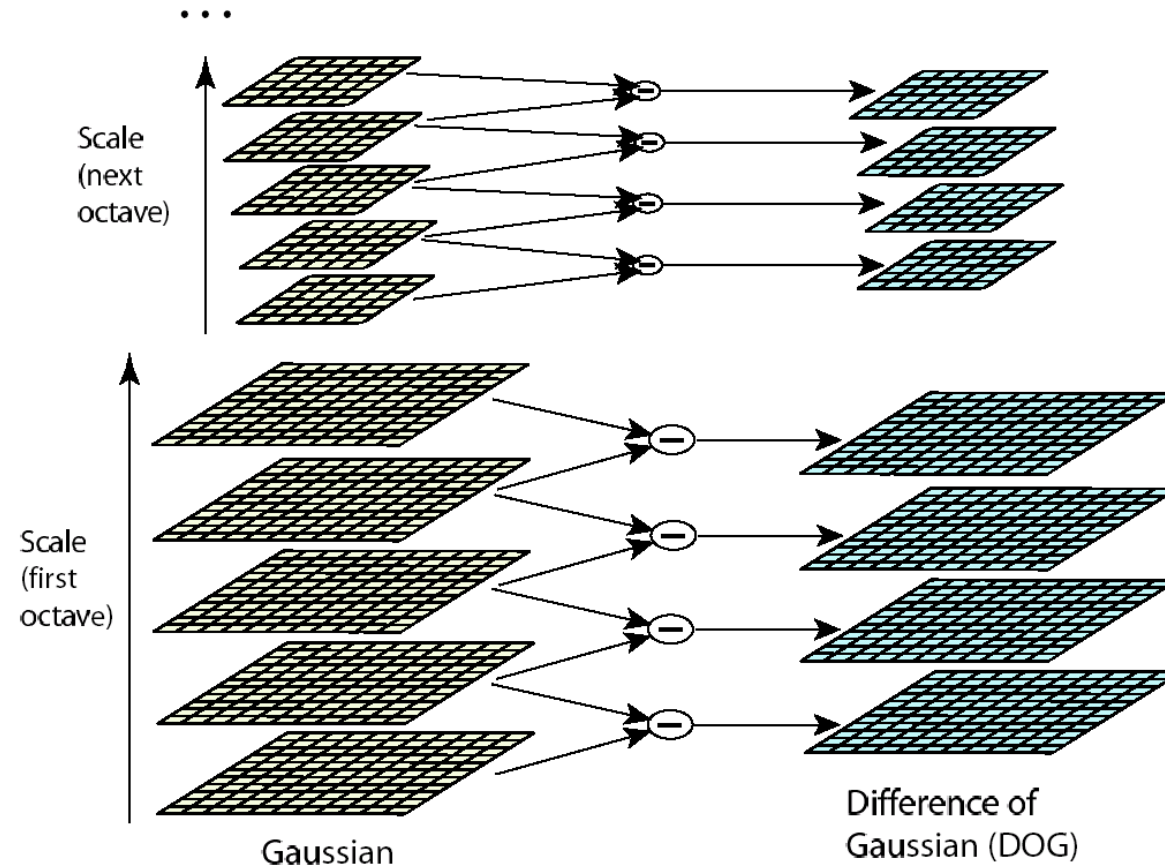
$$DoG = G(x, y, k\sigma) - G(x, y, \sigma)$$

(Difference of Gaussians)

*almost
same*



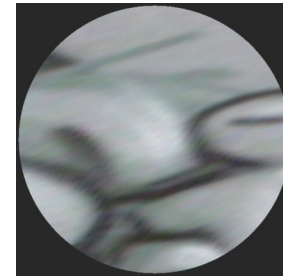
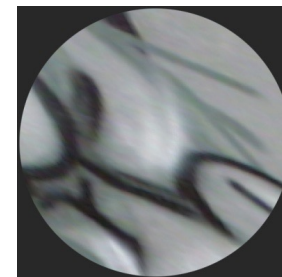
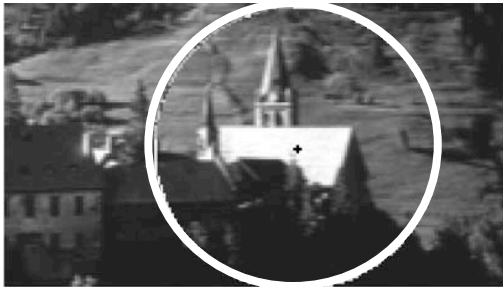
Efficient Implementation



David G. Lowe. "Distinctive image features from scale-invariant keypoints." *IJCV* 60 (2), pp. 91-110, 2004.

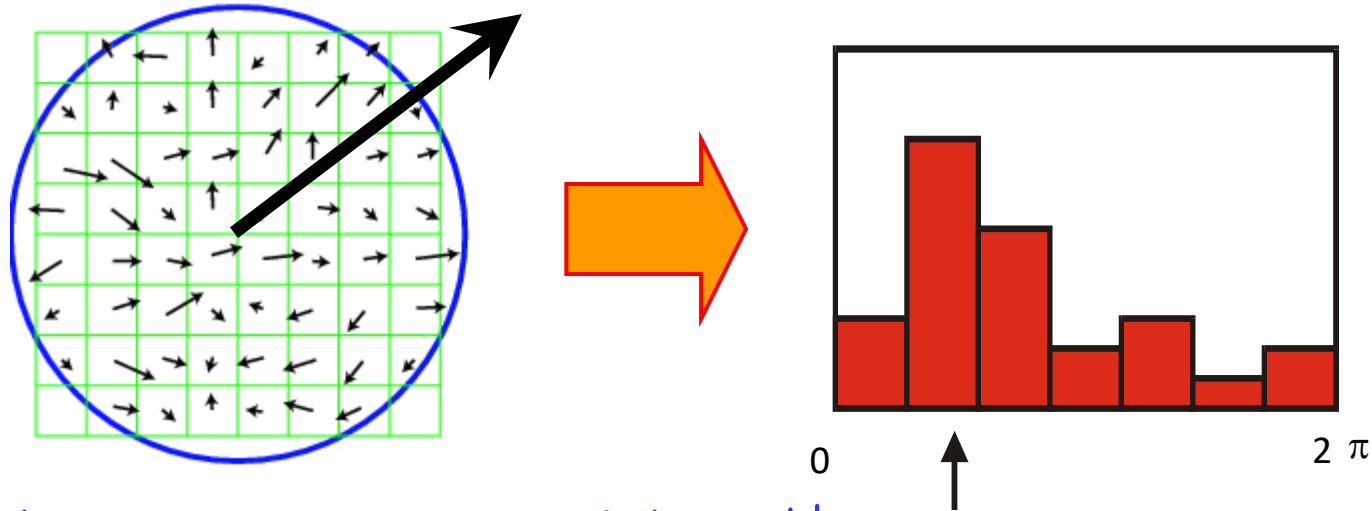
From Feature Detection to Feature Description

- Scaled and rotated versions of the same neighborhood will give rise to blobs that are related by the same transformation
- What to do if we want to compare the appearance of these image regions?
 - *Normalization*: transform these regions into same-size circles
 - Problem: rotational ambiguity



Eliminating Rotation Ambiguity

- To assign a unique orientation to circular image windows:
 - Create histogram of local gradient directions in the patch
 - Assign canonical orientation at peak of smoothed histogram



we know the location, scale, direction of the patch.

SIFT Features

Detected features with characteristic scales and orientations:



David G. Lowe. "Distinctive image features from scale-invariant keypoints." *IJCV* 60 (2), pp. 91-110, 2004.

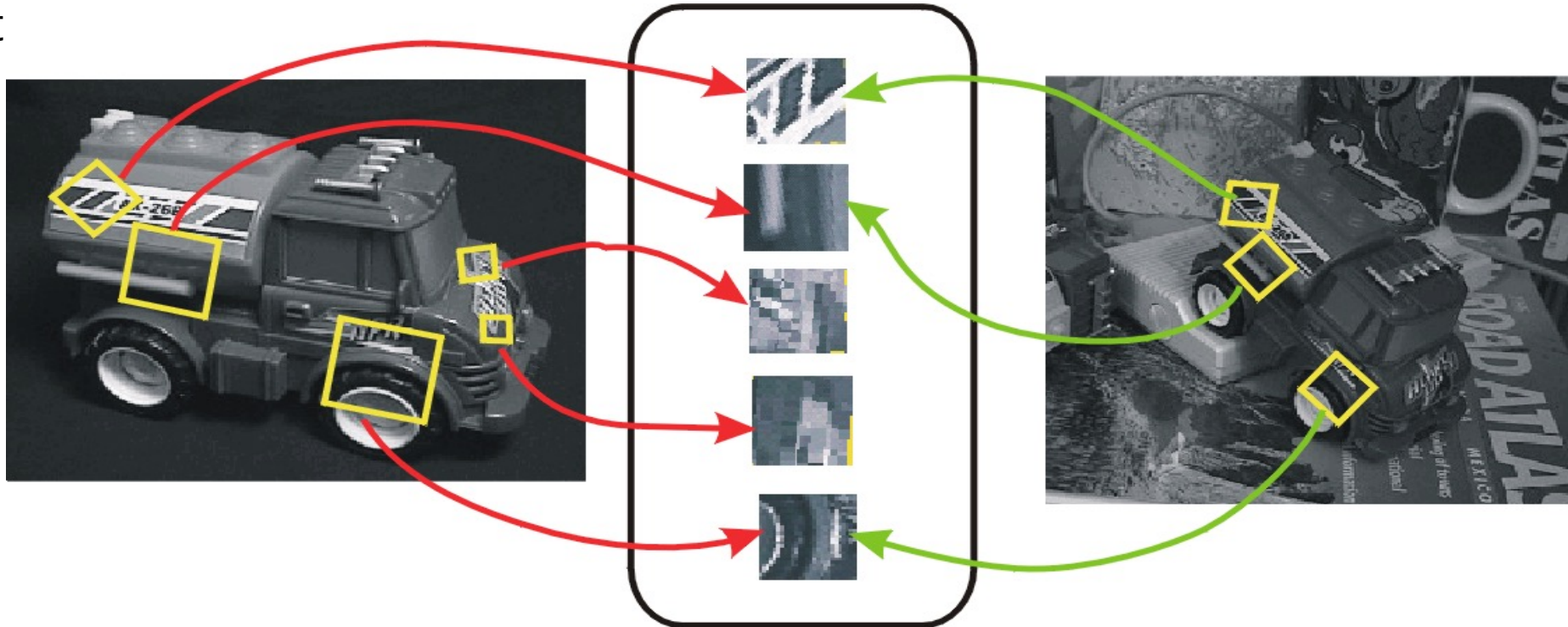
From Feature Detection to Feature Description

Detection is *covariant*:

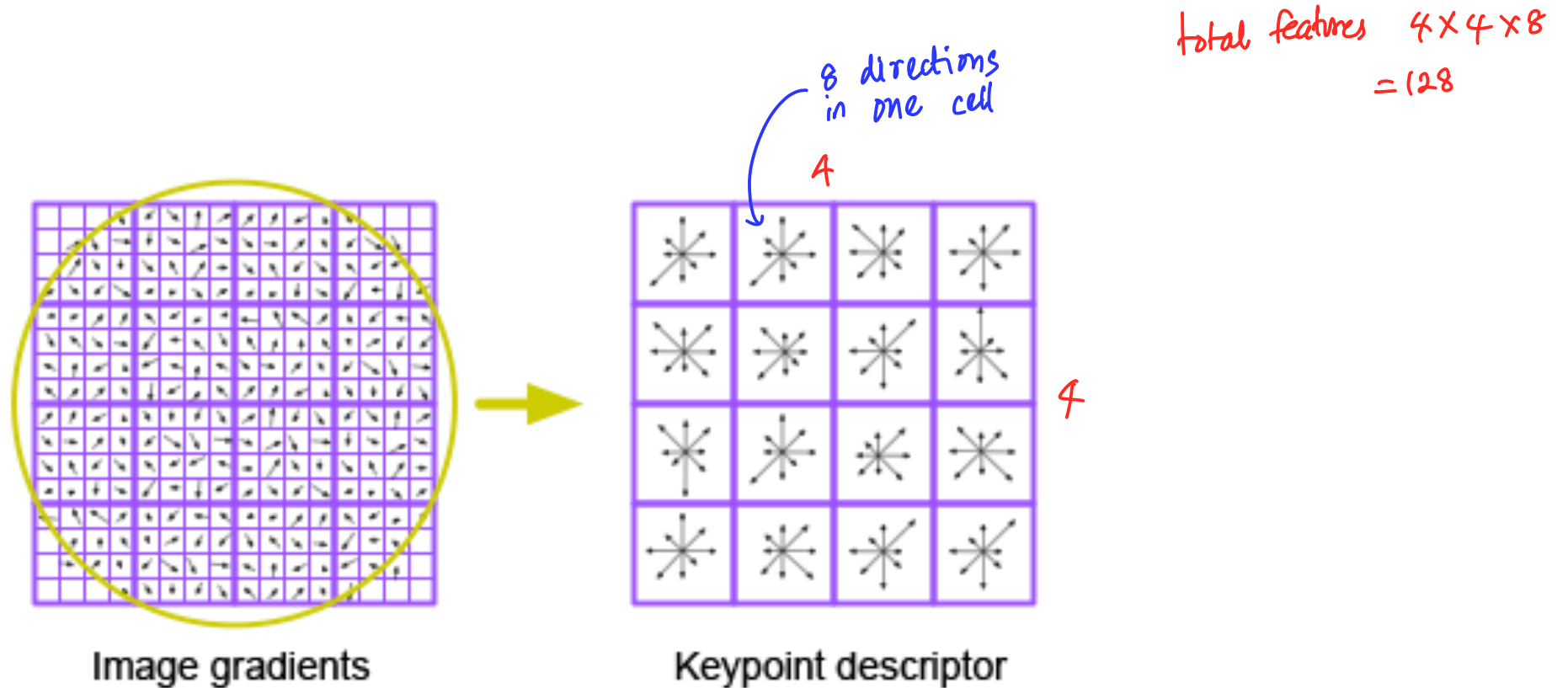
$$\text{features}(\text{transform}(\text{image})) = \text{transform}(\text{features}(\text{image}))$$

Description is *invariant*:

feat



SIFT Descriptors



David G. Lowe. "Distinctive image features from scale-invariant keypoints." *IJCV* 60 (2), pp. 91-110, 2004.

Properties of SIFT

Extraordinarily robust detection and description technique

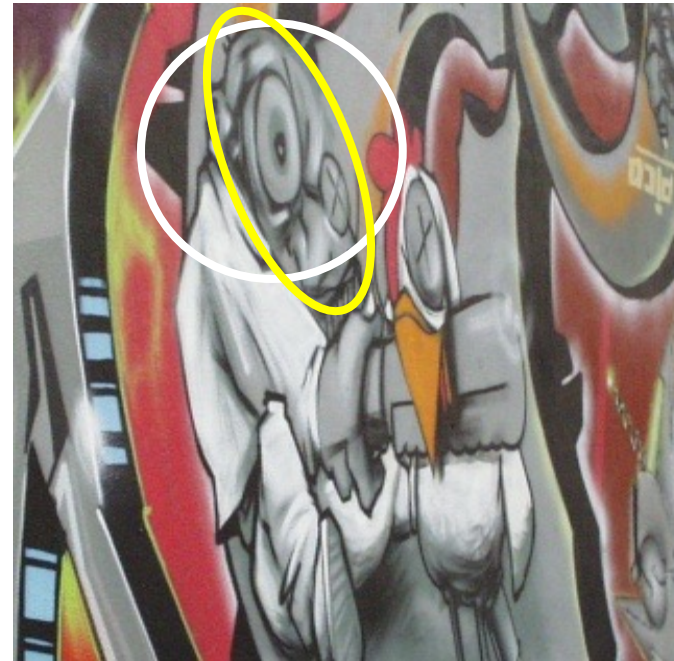
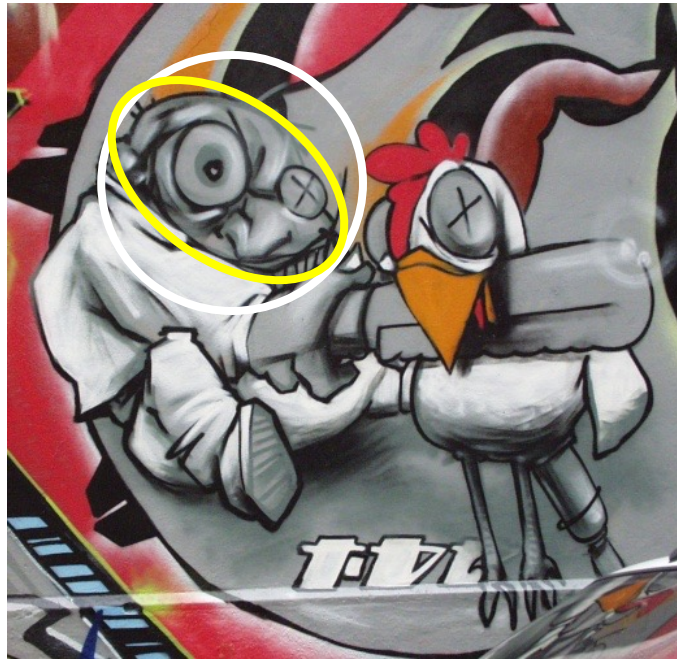
- Can handle changes in viewpoint
 - Up to about 60 degree out-of-plane rotation
- Can handle significant changes in illumination
 - Sometimes even day vs. night
- Fast and efficient—can run in real time
- Lots of code available



Source: N. Snavely

Affine adaptation

Affine transformation approximates viewpoint changes for roughly planar objects and roughly orthographic cameras



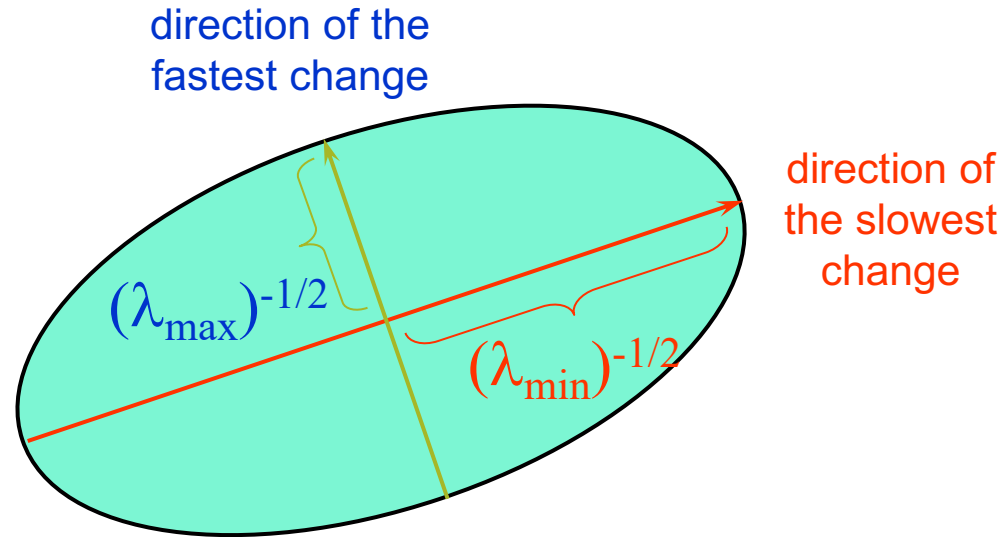
Affine Adaptation

Consider the second moment matrix of the window containing the blob:

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = R^{-1} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R$$

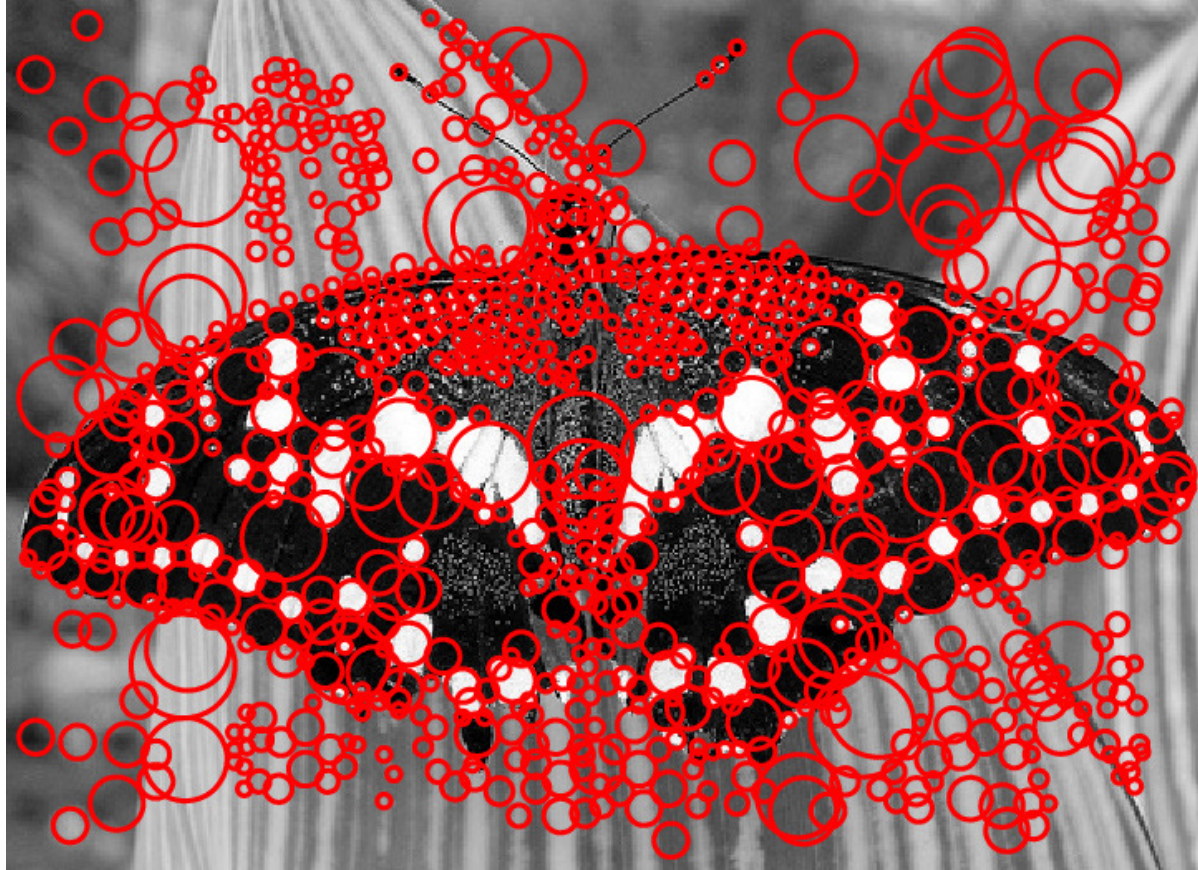
Recall:

$$\begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix} = \text{const}$$



This ellipse visualizes the “characteristic shape” of the window

Affine Adaptation Example



Scale-invariant regions (blobs)

Affine Adaptation Example



Affine-adapted blobs